Lecture 5

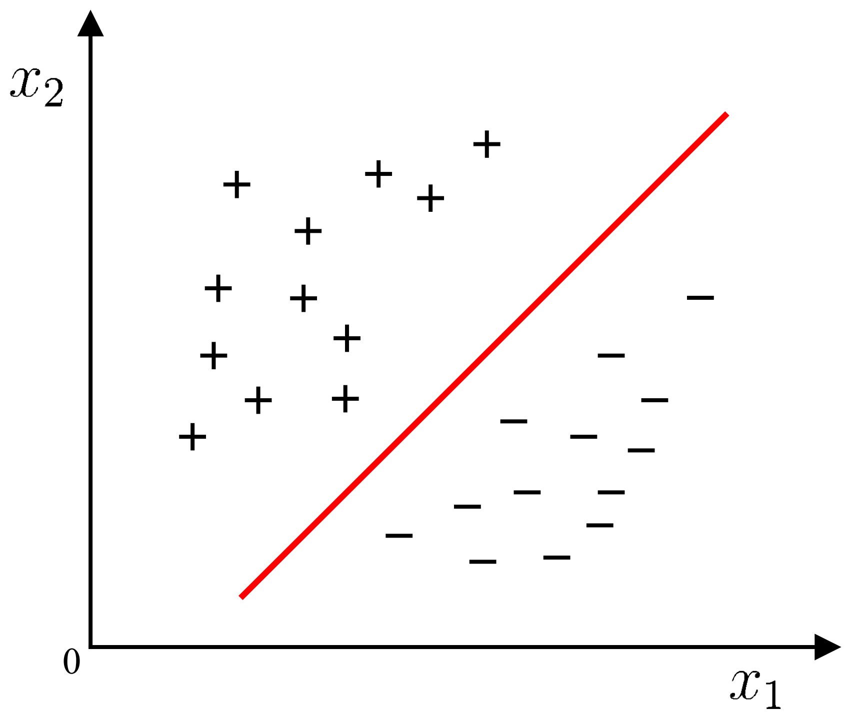
Support Vector Machines

Outline

- Margin and Support Vector
- Dual Problem
- Soft Margin and Regularization
- Kernel Function
- Support Vector Regression
- Kernel Methods

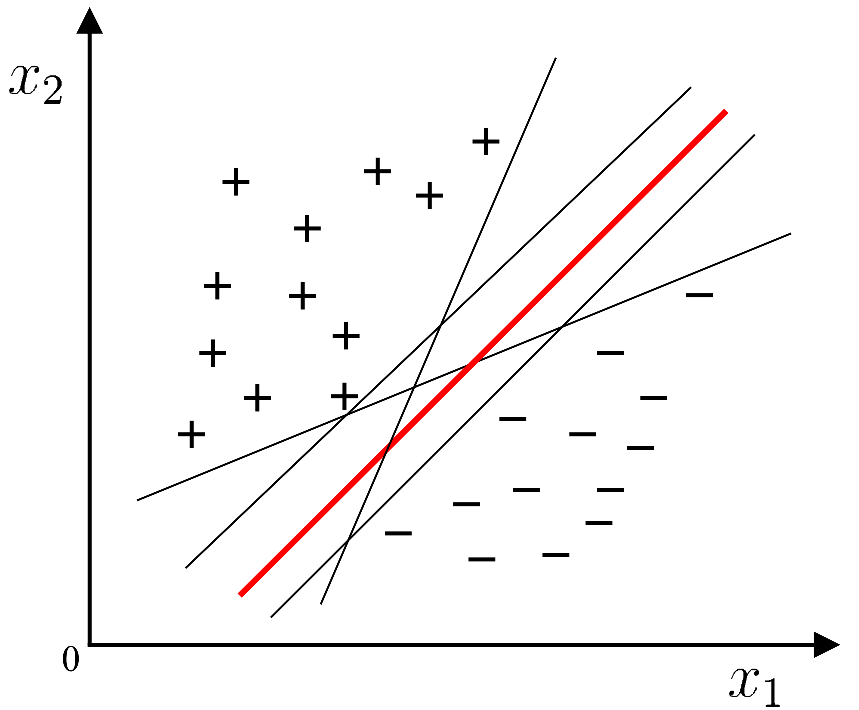
Introduction

Linear model： find a separating hyperplane in the sample space that can separate samples of different classes.



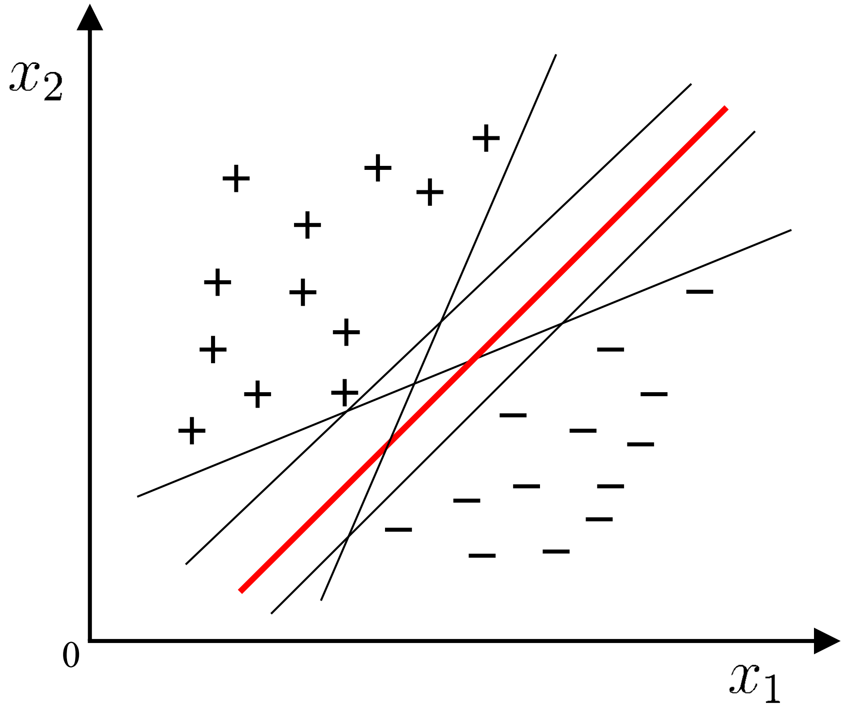
Introduction

-Q: There could be multiple qualified separating hyperplanes, which one should be chosen?



Introduction

-Q: There could be multiple qualified separating hyperplanes, which one should be chosen?



-A: The one right in the middle of two classes. It has the best tolerance to local data perturbation, the strongest generalization ability and the most robust classification results.

Support Vector Machine

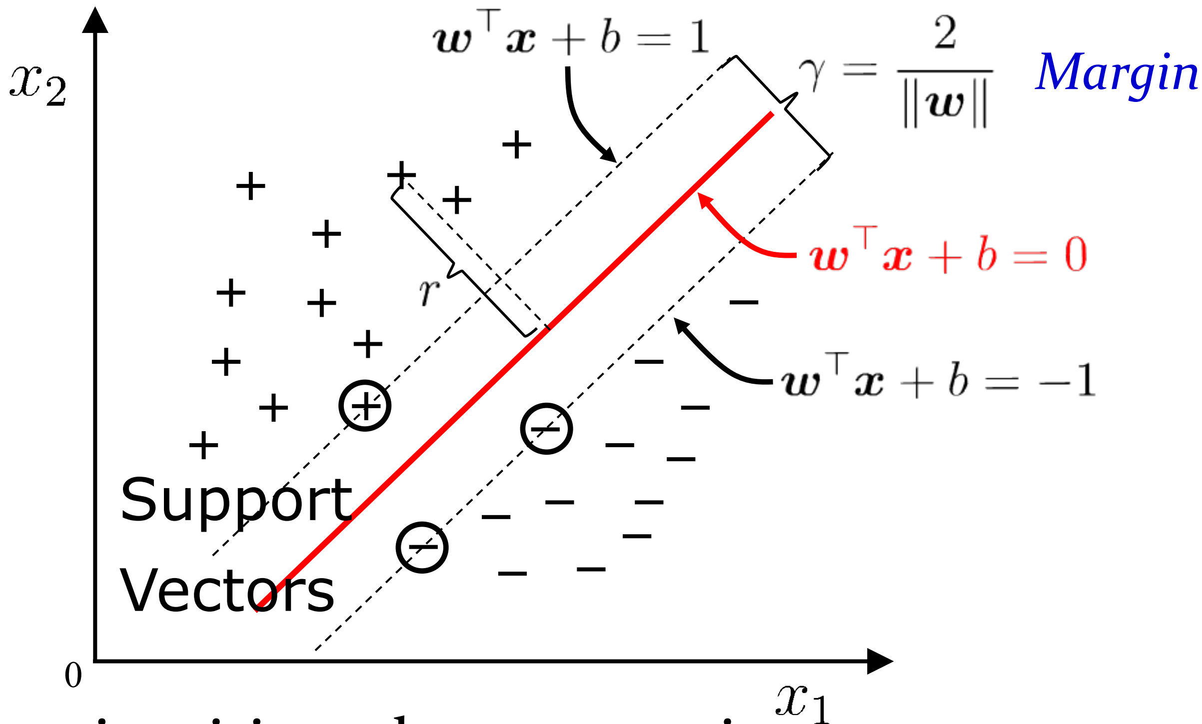
- Vladimir Vapnik
  - Born in the Soviet Union
    - PhD in statistics, 1964
    - Co-invented the VC dimension
      - Vapnik-Chervonenkis Theory, 1974
  - Moved to the U.S. in 1990
    - Jointed AT&T
    - Developed SVM algorithm in the 90's

Vladimir N. Vapnik
1936-Present

Margin and Support Vector

SVMs (Vapnik, 1990's) choose the linear separator with the largest margin.

Robust to outliers!

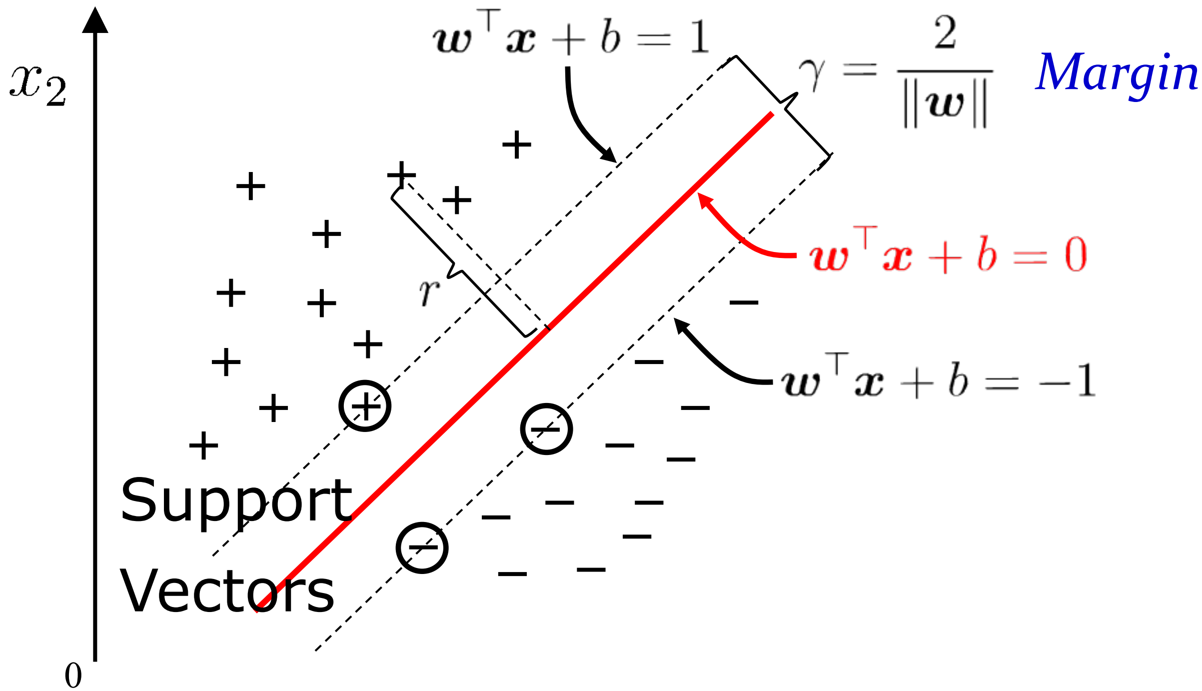


Good according to intuition, theory, practice.

SVM became famous when, using images as input, it gave accuracy comparable to neural-network with hand-designed features in a handwriting recognition task

Margin and Support Vector

SVMs (Vapnik, 1990's) choose the linear separator with the largest margin.



Robust to outliers!

注：由于点 x 到直线（超平面）$w^T x + b = 0$的距离具有缩放不变性，SVM 通过令 $\min_i y_i(\mathbf{w}^T\mathbf{x}_i + b) = 1$ 进行标准化。

证明 w 垂直于超平面

步骤 1：取超平面上的任意两点

设 $\mathbf{x}_1$ 和 $\mathbf{x}_2$ 是超平面上的两个点，满足：

$$\mathbf{w} \cdot \mathbf{x}_1 + b = 0, \quad \mathbf{w} \cdot \mathbf{x}_2 + b = 0.$$

步骤 2：构造超平面内的向量

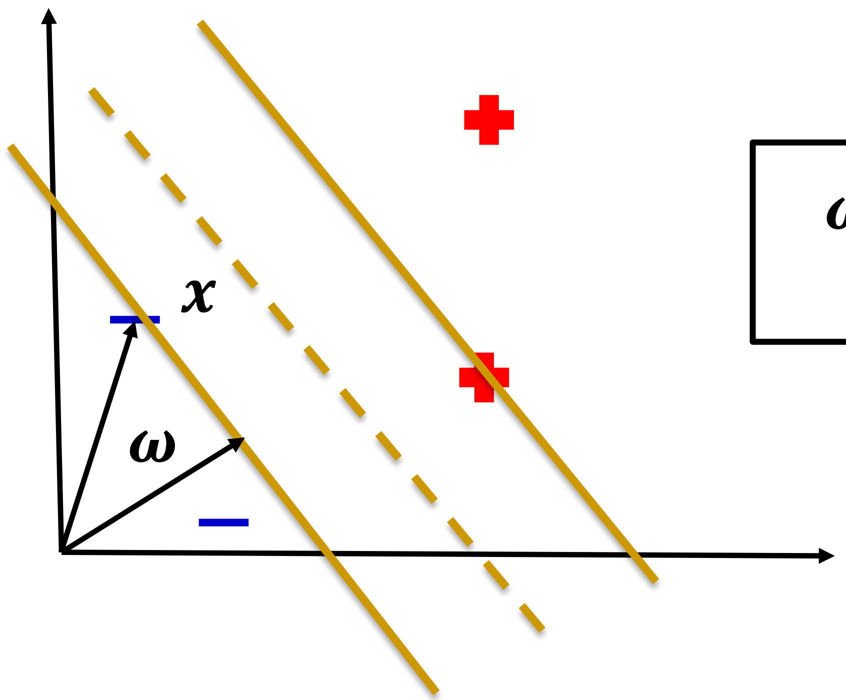
两点之间的向量 $\mathbf{v} = \mathbf{x}_1 - \mathbf{x}_2$ 位于超平面内。计算 $\mathbf{w}$ 与 $\mathbf{v}$ 的点积：

$$\mathbf{w} \cdot \mathbf{v} = \mathbf{w} \cdot (\mathbf{x}_1 - \mathbf{x}_2) = \mathbf{w} \cdot \mathbf{x}_1 - \mathbf{w} \cdot \mathbf{x}_2 = (-b) - (-b) = 0.$$

步骤 3：正交性结论

由于 $\mathbf{w} \cdot \mathbf{v} = 0$，说明 $\mathbf{w}$ 与超平面内的任意向量 $\mathbf{v}$ 正交。因此，$\mathbf{w}$ 是超平面的法向量，方向垂直于超平面。

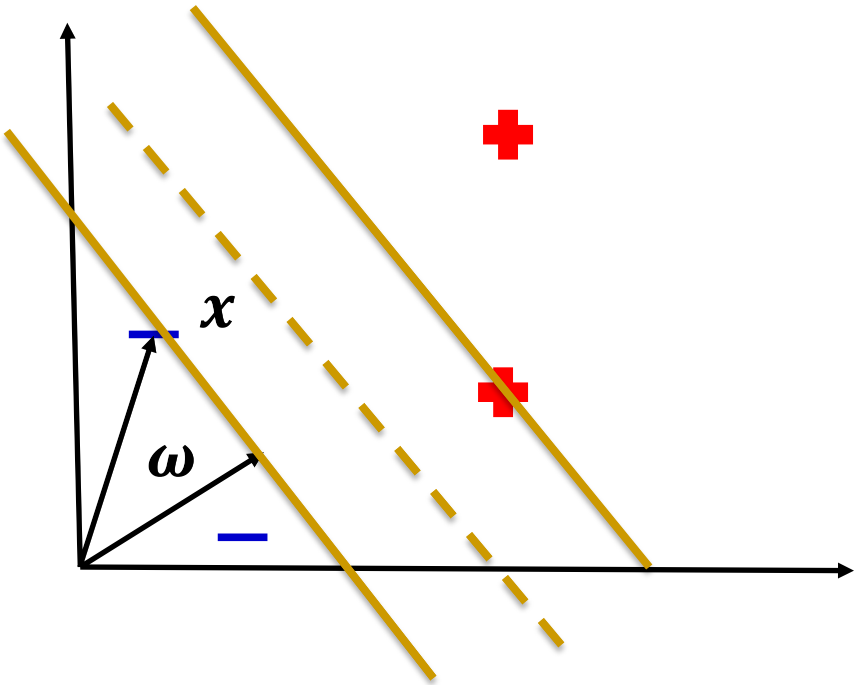
Decision Boundary - SVM



$$\boldsymbol{\omega} \cdot \boldsymbol{x} + \boldsymbol{b} \geq 0, \textit{ then class} +$$

Decision Rule

Decision Boundary - SVM

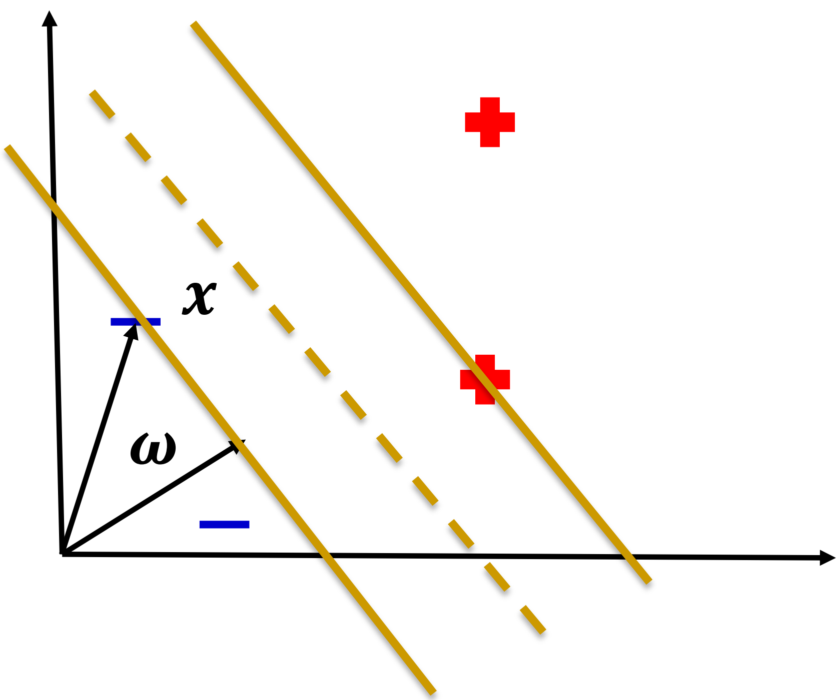


$$\boldsymbol{\omega} \cdot \boldsymbol{x}_{+} + \boldsymbol{b} \geq 1, \quad \textit{for class +}$$
$$\boldsymbol{\omega} \cdot \boldsymbol{x}_{-} + \boldsymbol{b} \leq -1, \textit{for class -}$$

y_i such that: $y_i = +1$ for class +

$y_i = -1$ for class -

Decision Boundary - SVM



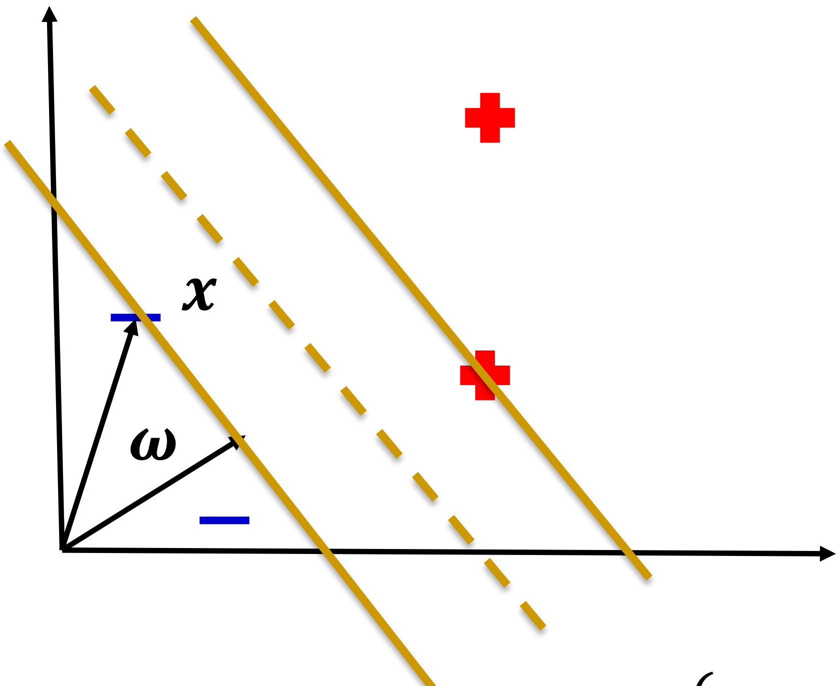
$$\boldsymbol{\omega} \cdot \boldsymbol{x}_+ + \boldsymbol{b} \geq 1, \quad \textit{for class} +$$
$$\boldsymbol{\omega} \cdot \boldsymbol{x}_- + \boldsymbol{b} \leq -1, \textit{for class} -$$

y_i such that: $y_i = +1$ for class +

$y_i = -1$ for class -

$$y_i(\boldsymbol{\omega} \cdot \boldsymbol{x}_i + \boldsymbol{b}) \geq 1, \quad \textit{for class} +$$
$$y_i(\boldsymbol{\omega} \cdot \boldsymbol{x}_i + \boldsymbol{b}) \geq 1, \quad \textit{for class} -$$

Decision Boundary - SVM



$$\boldsymbol{\omega} \cdot \boldsymbol{x}_+ + \boldsymbol{b} \geq 1, \quad \textit{for class +}$$
$$\boldsymbol{\omega} \cdot \boldsymbol{x}_- + \boldsymbol{b} \leq -1, \textit{for class -}$$

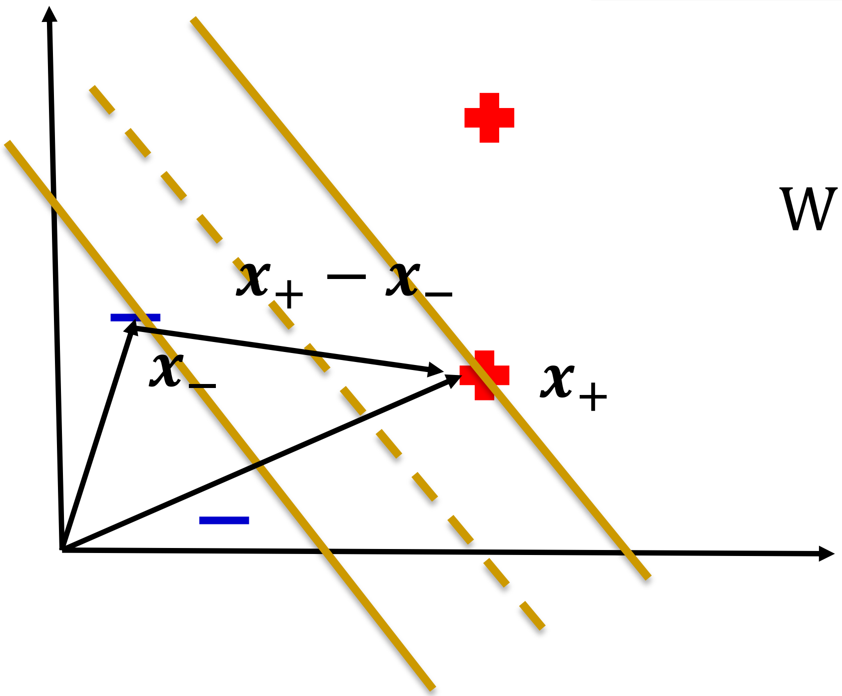
$$y_i \text{ such that: } y_i = +1 \text{ for class +}$$
$$y_i = -1 \text{ for class -}$$

$$y_i(\boldsymbol{\omega} \cdot \boldsymbol{x}_i + \boldsymbol{b}) \geq 1, \quad \textit{for class +}$$
$$y_i(\boldsymbol{\omega} \cdot \boldsymbol{x}_i + \boldsymbol{b}) \geq 1, \quad \textit{for class -}$$

$$y_i(\boldsymbol{\omega} \cdot \boldsymbol{x}_i + \boldsymbol{b}) - 1 \geq 0,$$
$$y_i(\boldsymbol{\omega} \cdot \boldsymbol{x}_i + \boldsymbol{b}) - 1 = 0, \textit{for boundary cases}$$

Decision Boundary - SVM

$$\boxed{y_i(\boldsymbol{\omega} \cdot \boldsymbol{x}_i + \boldsymbol{b}) - 1 = 0, \textit{ for support vectors}}$$



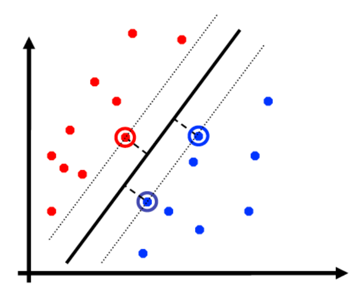
$$Width = (\boldsymbol{x}_+ - \boldsymbol{x}_-) \cdot \frac{\boldsymbol{\omega}}{||\boldsymbol{\omega}||} = \frac{2}{||\boldsymbol{\omega}||}$$

$1 - \boldsymbol{b}$ $\quad$ $1 + \boldsymbol{b}$

$$\max \frac{2}{||\boldsymbol{\omega}||} \Leftrightarrow \max \frac{1}{||\boldsymbol{\omega}||} \Leftrightarrow min||\boldsymbol{\omega}|| \Leftrightarrow min \frac{1}{2}||\boldsymbol{\omega}||^2$$

Support Vector Machines: 3 key ideas

- Use **optimization** to find solution (i.e. a hyperplane) with few errors
- Seek **large margin** separator to improve generalization
- Use **kernel trick** to make large feature spaces computationally efficient



The Primal Form of SVM

Maximum margin: finding the parameters $\boldsymbol{w}$ and b that maximize

$$\begin{aligned} \mathbf{w}\cdot\mathbf{x}_i + b \geq 1, &\quad \text{if } y_i = +1; \\ \mathbf{w}\cdot\mathbf{x}_i + b \leq -1, &\quad \text{if } y_i = -1 \end{aligned}$$

$$\begin{aligned} \arg\max_{\boldsymbol{w},b} \quad & \frac{2}{\|\boldsymbol{w}\|} \\ \text{s.t.} \quad & y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1, \; i = 1, 2, \ldots, m. \end{aligned}$$

$$\begin{aligned} \arg\min_{\boldsymbol{w},b} \quad & \frac{1}{2}\|\boldsymbol{w}\|^2 \\ \text{s.t.} \quad & y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1, \; i = 1, 2, \ldots, m. \end{aligned}$$

凸二次规划（QP）问题，可直接用优化算法求解

This is an optimization problem with linear, inequality constraints.

Review of multivariable calculus

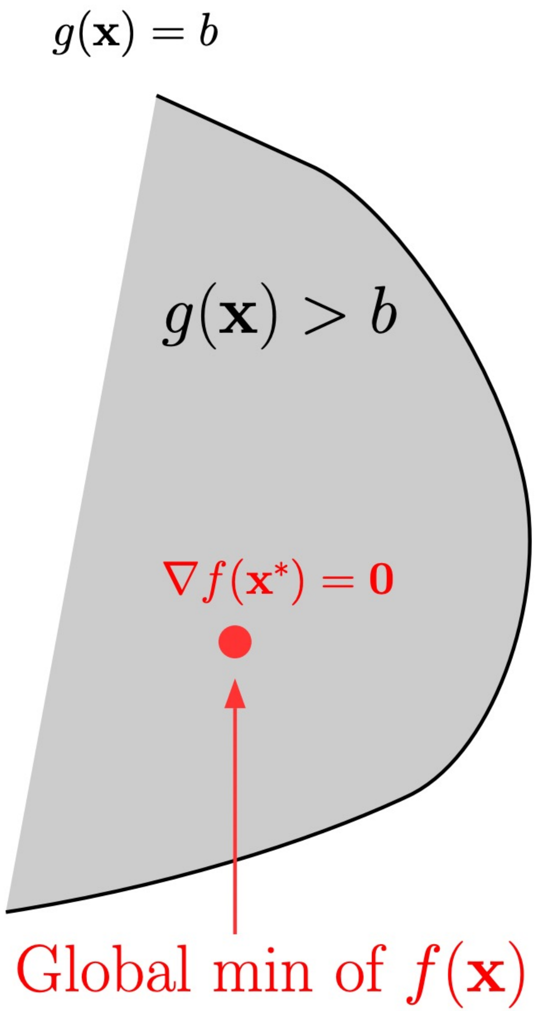
Consider the following constrained optimization problem

$$\min f(\mathbf{x}) \qquad \text{subject to} \quad g(\mathbf{x}) \geq b$$

There are two cases regarding where the global minimum of f(x) is attained:

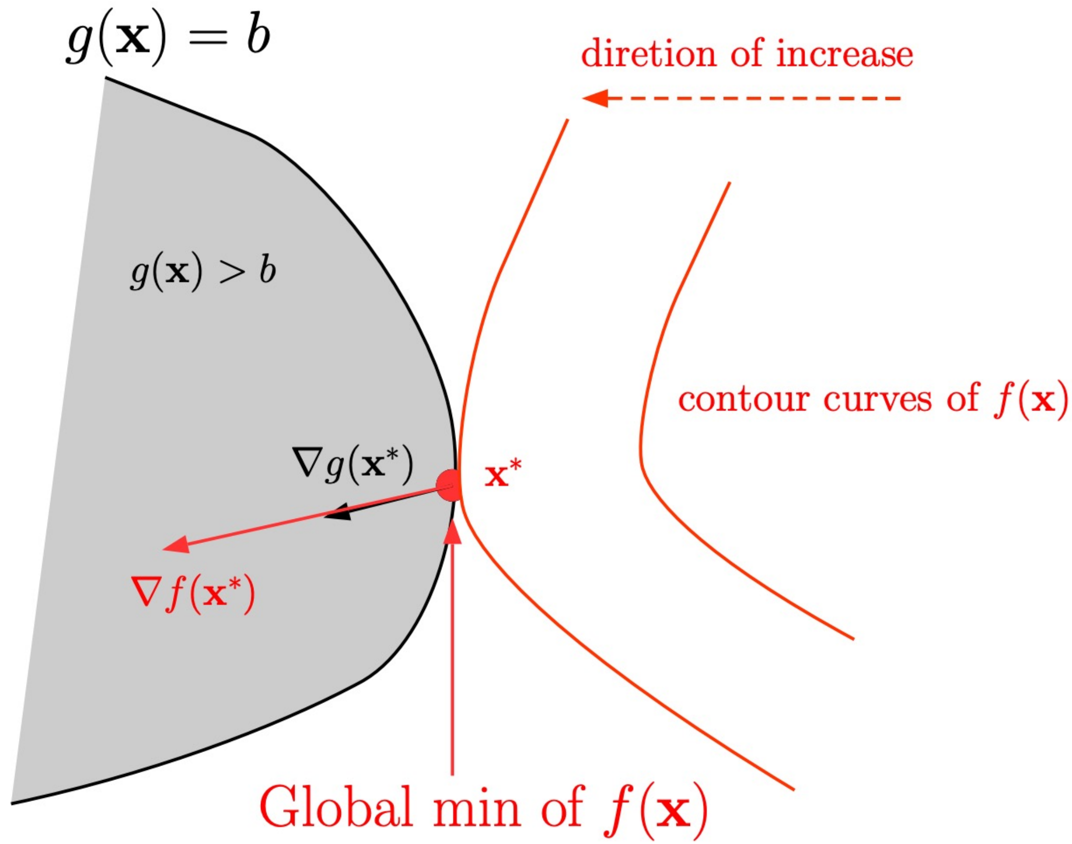
(1) At an interior point x^* $(i.e., g(x^*) > b)$. In this case x^* is just a critical point of $f(x)$.

The Lagrange Method



The Lagrange Method

(2) At a boundary point x^* (i.e., $g(x^*) = b$). In this case, there exists a constant $\lambda > 0$ such that $\nabla f(x^*) = \lambda \cdot \nabla g(x^*)$.



Why?

An Example

Consider minimizing $f(x, y) = x^2 + y^2$, subject to the constraint $g(x, y) = x + y \geq 1$.

- The feasible region is defined by $x + y \geq 1$, and the boundary is the line $x + y = 1$.
- At the boundary point, the gradient of f, $\nabla f = (2x, 2y)$, must align with the gradient of $g, \nabla g = (1,1)$. This means: $(2x, 2y) = \lambda(1,1) \Rightarrow x = y = \lambda/2$.
- Combining this with the constraint $x + y = 1$, we solve for $\lambda = 1$. Thus, the optimal point is $(\frac{1}{2}, \frac{1}{2})$.

The Lagrange Method

The above two cases are unified by the **method of Lagrange multipliers**:

- Form the Lagrange function

$$L(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda(g(\mathbf{x}) - b)$$

- Find all critical points by solving

$$\begin{aligned} \nabla_{\mathbf{x}} L = \mathbf{0}: \quad \nabla f(\mathbf{x}) &= \lambda \nabla g(\mathbf{x}) \\ \lambda(g(\mathbf{x}) - b) &= 0 \\ \lambda &\geq 0 \\ g(\mathbf{x}) &\geq b \end{aligned}$$

Remark. The solutions give all candidate points for the global minimizer (one needs to compare them and pick the best one).

The Lagrange Method

Remarks:

• The above equations are called Karush-Kuhn-Tucker (**KKT**) conditions.

• When there are multiple inequality constraints

$$\min f(\mathbf{x}) \qquad \text{subject to} \quad g_1(\mathbf{x}) \geq b_1, \ldots, g_k(\mathbf{x}) \geq b_k$$

the method works very similarly:

The Lagrange Method

– Form the Lagrange function

$$L(\mathbf{x}, \lambda_1, \ldots, \lambda_k) = f(\mathbf{x}) - \lambda_1(g_1(\mathbf{x}) - b_1) - \cdots - \lambda_k(g_k(\mathbf{x}) - b_k)$$

– Find all critical points by solving

$$\begin{aligned} \nabla_{\mathbf{x}} L = \mathbf{0} : \quad \frac{\partial L}{\partial x_1} = 0, \ldots, \frac{\partial L}{\partial x_n} = 0 \\ \lambda_1(g_1(\mathbf{x}) - b_1) = 0, \ldots, \lambda_k(g_k(\mathbf{x}) - b_k) = 0 \\ \lambda_1 \geq 0, \ldots, \lambda_k \geq 0 \\ g_1(\mathbf{x}) \geq b_1, \ldots, g_k(\mathbf{x}) \geq b_k \end{aligned}$$

and compare them to pick the best one.

The Lagrange Method

Consider a general optimization problem (called as primal problem)

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & g_i(x) \geq 0, i = 1, \cdots, k \\ & h_j(x) = 0, j = 1, \cdots, m. \end{aligned}$$

We define its Lagrangian as

$$L(x, u, v) = f(x) - \sum_{i=1}^{k} \lambda_i g_i(x) + \sum_{j=1}^{m} u_j h(x)$$

Lagrangian multipliers $\lambda \in \mathbb{R}^k, u \in \mathbb{R}^m$.

The Lagrange Method

Lemma 1 *At each feasible* x, $f(x) = \sup_{\lambda \geq 0,\, u} L(x, \lambda, u)$, *and the supremum is taken iff* $\lambda \geq 0$ *satisfying* $\lambda_i g_i(x) = 0, i = 1, \dots, k$.

Proof: At each feasible x, we have $g_i(x) \geq 0$ and $h(x) = 0$, thus $L(x, \lambda, u) = f(x) - \sum_{i=1}^{k} \lambda_i g_i(x) + \sum_{j=1}^{m} u_j h_j(x) \leq f(x)$.

Proposition 2 *The optimal value of primal problem, named as* f^*, *satisfies:*

$$f^* = \inf_{x} \sup_{\lambda \geq 0,\, u} L(x, \lambda, u)$$

Proof: First considering feasible x (marked as $x \in C$), we have $f^* = \inf_{x \in C} f(x) = \inf_{x} \sup_{\lambda \geq 0,\, u} L(x, \lambda, u)$. Second considering non-feasible x, since $\sup_{\lambda \geq 0,\, u} L(x, \lambda, u) = \propto$ for any $x \notin C$, $\inf_{x \notin C} \sup_{\lambda \geq 0,\, u} L(x, \lambda, u) = \propto$. In total, $f^* = \inf_{x} \sup_{\lambda \geq 0,\, u} L(x, \lambda, u)$.

The Dual Problem

A re-written Primal Problem :

$$\min_{x} \max_{\lambda \geq 0, u} L(x, \lambda, u)$$

The Dual Problem:

$$\max_{\lambda \geq 0, u} \min_{x} L(x, \lambda, u)$$

Although the primal problem is not required to be convex, the dual problem is always convex.

Theorem (weak duality):

Why?

$$d^* = \max_{\lambda \geq 0, u} \min_{x} L(x, \lambda, u) \leq \min_{x} \max_{\lambda \geq 0, u} L(x, \lambda, u) = p^*$$

Theorem (strong duality, *e.g.*, ***Slater's condition*****):**
If the primal is a convex problem, and there exists at least one strictly feasible $\tilde{x}$, meaning that $\exists \tilde{x}, g_i(\tilde{x}) > 0, i = 1, \ldots, k,\ h_j(\tilde{x}) = 0, j = 1, \ldots, m.$

$$d^* = p^*$$

- 对于任意固定的 $\lambda' \geq 0$ 和 u'，以及对任意 x，有：

$$d(\lambda', u') = \min_{x} L(x, \lambda', u') \leq L(x, \lambda', u')$$

- 同时，对于固定的 x，有：

$$L(x, \lambda', u') \leq \max_{\lambda \geq 0, u} L(x, \lambda, u)$$

- 因此，结合两者：

$$d(\lambda', u') \leq \max_{\lambda \geq 0, u} L(x, \lambda, u) \quad \text{对所有 } x$$

- 既然 $d(\lambda', u')$ 小于等于右边对于每个 x 的值，那么它也必须小于等于右边的最小值：

$$d(\lambda', u') \leq \min_{x} \max_{\lambda \geq 0, u} L(x, \lambda, u)$$

- 由于 λ' 和 u' 是任意的，上述不等式对所有乘子成立。因此，对偶函数的最大值也满足：

$$\max_{\lambda \geq 0, u} d(\lambda, u) \leq \min_{x} \max_{\lambda \geq 0, u} L(x, \lambda, u)$$

Karush–Kuhn–Tucker (KKT) conditions

Necessary conditions

If x^* and λ^*, u^* are the primal and dual solutions respectively with zero duality gap, we will show that x^*, λ^*, u^* satisfy the KKT conditions.

$$f(x^*) = d(\lambda^*, u^*) \text{ by zero duality gap assumption}$$

$$= \min_x f(x) - \sum_{i=1}^{k} \lambda_i^* g_i(x) + \sum_{j=1}^{m} u_j^* h_j(x), \text{ by definition}$$

stationarity

$$\leq f(x^*) - \sum_{i=1}^{k} \lambda_i^* g_i(x^*) + \sum_{j=1}^{m} u_j^* h_j(x^*)$$

⇨ equality: x^* minimizes $\mathrm{L}(x, \lambda^*, u^*)$

$$\leq f(x^*)$$

⇨ equality: $\lambda_i^* \, g_i(x^*) = 0$

complementary slackness

For convex problems with strong duality (e.g., when Slater's condition is satisfied), the KKT conditions are necessary and sufficient optimality conditions, i.e., x^* and (λ^*, u^*) are primal and dual optimal if and only if the KKT conditions hold.

Outline

- Margin and Support Vector
- Dual Problem
- Soft Margin and Regularization
- Kernel Function
- Support Vector Regression
- Kernel Methods

The Primal Form of SVM

Maximum margin: finding the parameters $\boldsymbol{w}$ and b that maximize

$$\begin{aligned} \mathbf{w} \cdot \mathbf{x}_i + b \geq 1, &\quad \text{if } y_i = +1; \\ \mathbf{w} \cdot \mathbf{x}_i + b \leq -1, &\quad \text{if } y_i = -1 \end{aligned}$$

$$\Updownarrow$$

$$\begin{aligned} \arg\max_{\boldsymbol{w},b} \quad & \frac{2}{\|\boldsymbol{w}\|} \\ \text{s.t.} \quad & y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1, \ i = 1, 2, \ldots, m. \end{aligned}$$

$$\Downarrow$$

$$\begin{aligned} \arg\min_{\boldsymbol{w},b} \quad & \frac{1}{2}\|\boldsymbol{w}\|^2 \\ \text{s.t.} \quad & y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1, \ i = 1, 2, \ldots, m. \end{aligned}$$

This is an optimization problem with linear, inequality constraints.

Dual problem

- Lagrange multipliers
  - Step-1： introducing a Lagrange multiplier $\alpha_i \geq 0$, gives the Lagrange function

$$L(\boldsymbol{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\boldsymbol{w}\|^2 - \sum_{i=1}^{m} \alpha_i \left(y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) - 1\right)$$

  - Step-2： Setting the partial derivatives of $L(\boldsymbol{w}, b, \boldsymbol{\alpha})$ with respect to $\boldsymbol{w}$ and b to 0 gives

$$\boldsymbol{w} = \sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i, \quad \sum_{i=1}^{m} \alpha_i y_i = 0.$$

  - Step-3： Substituting back

$$\begin{aligned} \min_{\boldsymbol{\alpha}} \quad & \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \boldsymbol{x}_i^\top \boldsymbol{x}_j - \sum_{i=1}^{m} \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^{m} \alpha_i y_i = 0, \ \alpha_i \geq 0, \ i = 1, 2, \ldots, m. \end{aligned}$$

Sparsity of the solution

- desired model: $f(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x} + b = \boxed{\sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i^\top \boldsymbol{x}} + b$

- KKT conditions:

$$\boldsymbol{w} = \sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i, \quad \sum_{i=1}^{m} \alpha_i y_i = 0.$$ stationarity

$$\begin{cases} \alpha_i \geq 0, \\ y_i f(\boldsymbol{x}_i) \geq 1, \\ \alpha_i (y_i f(\boldsymbol{x}_i) - 1) = 0. \end{cases}$$

dual constraints

primal constraints

complementary slackness

$y_i f(\boldsymbol{x}_i) > 1 \Rightarrow \alpha_i = 0$

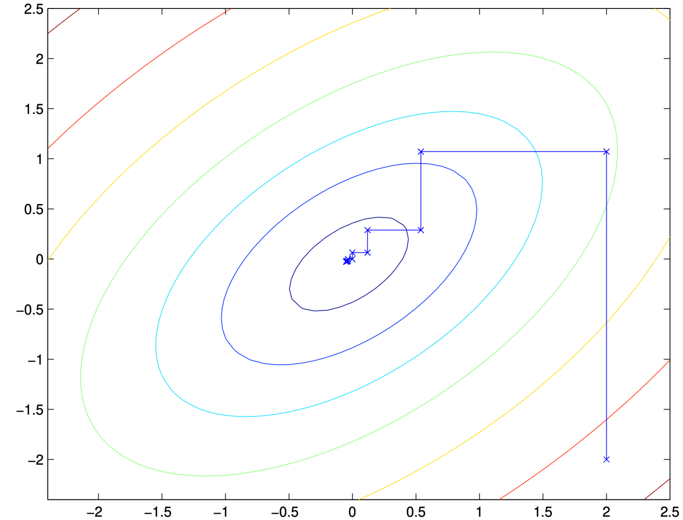
Sparsity of the solution of SVM: once the training completed, most training samples are no longer needed since the final model only depends on the support vectors.

Solving QP problem- Coordinate Ascent

$$\max_\alpha \quad W(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle.$$

$$\text{s.t.} \quad 0 \leq \alpha_i \qquad i = 1, \ldots, n$$

$$\sum_{i=1}^{n} \alpha_i y^{(i)} = 0.$$



Loop until convergence: $\{$

For $i = 1, \ldots, n$, $\{$

$$\alpha_i := \arg\max_{\hat{\alpha}_i} W(\alpha_1, \ldots, \alpha_{i-1}, \hat{\alpha}_i, \alpha_{i+1}, \ldots, \alpha_n).$$

$\}$

$\}$

Solving QP problem- SMO

- Basic idea： repeats the following two steps until convergence. blocked coordinate descent
  - Step1： Select two variables to be updated: α_i and α_j
  - Step2： Fix all the parameters and solve dual problem to update α_i and α_j
- If we only consider α_i and α_j, then we can rewrite the constraints in dual problem as

$$\alpha_i y_i + \alpha_j y_j = -\sum_{k \neq i,j} \alpha_k y_k, \quad \alpha_i \geq 0, \quad \alpha_j \geq 0.$$

Eliminate the variable with another and substitute back to the dual problem leads to a univariate quadratic programming problem, which has closed-form solutions. We "clip" the value of α to respect the constraints.

- Bias term b： determined by support vectors

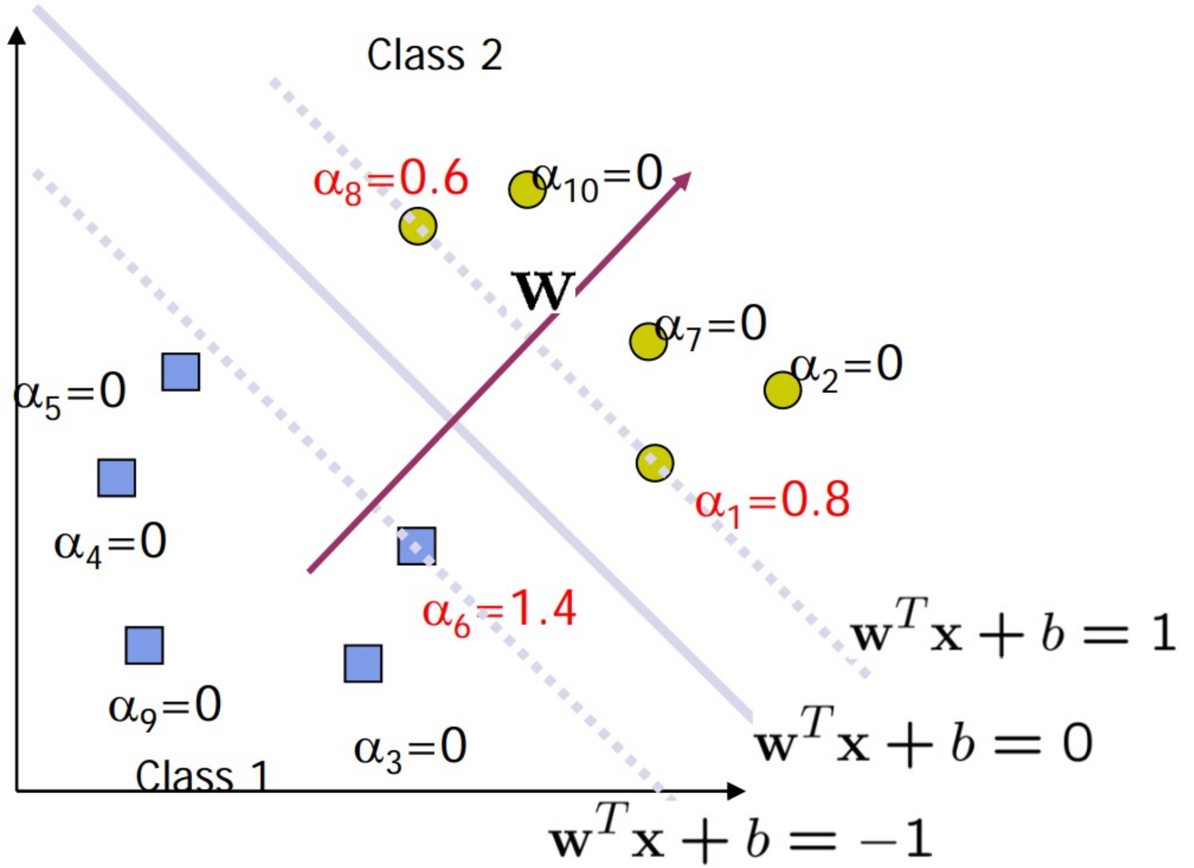
Solving QP problem- SMO

Repeat until convergence {

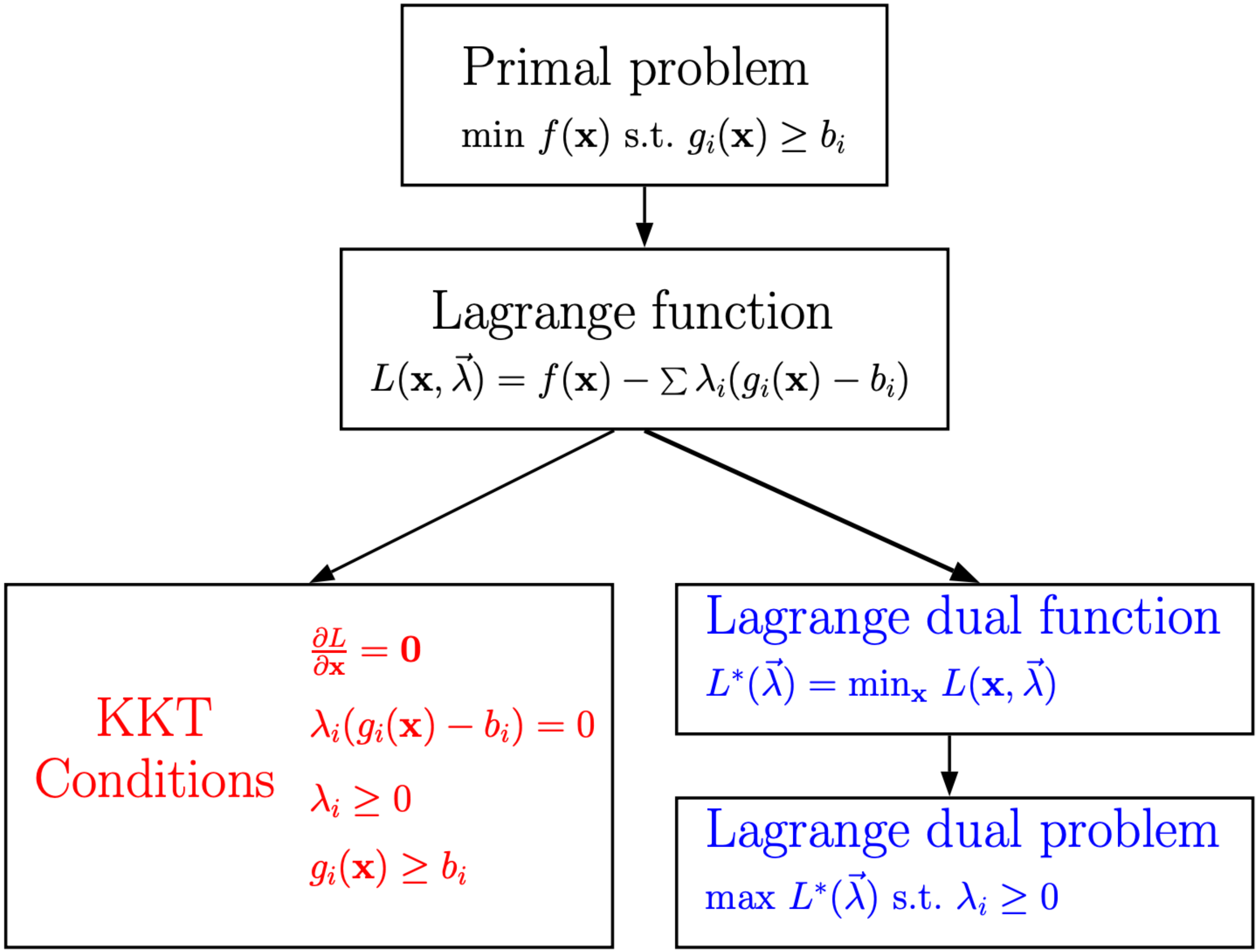
1. Heuristically choose a pair of α_i and α_j
2. Keeping all other α's fixed, optimize $W(\alpha)$ with respect to α_i and α_j.

}

Support vectors



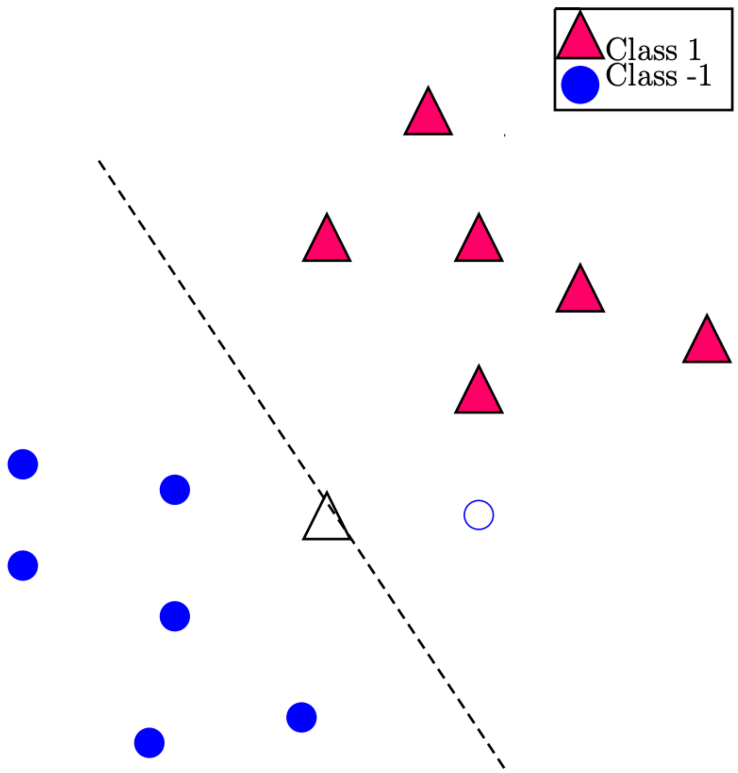
The Lagrange dual problem

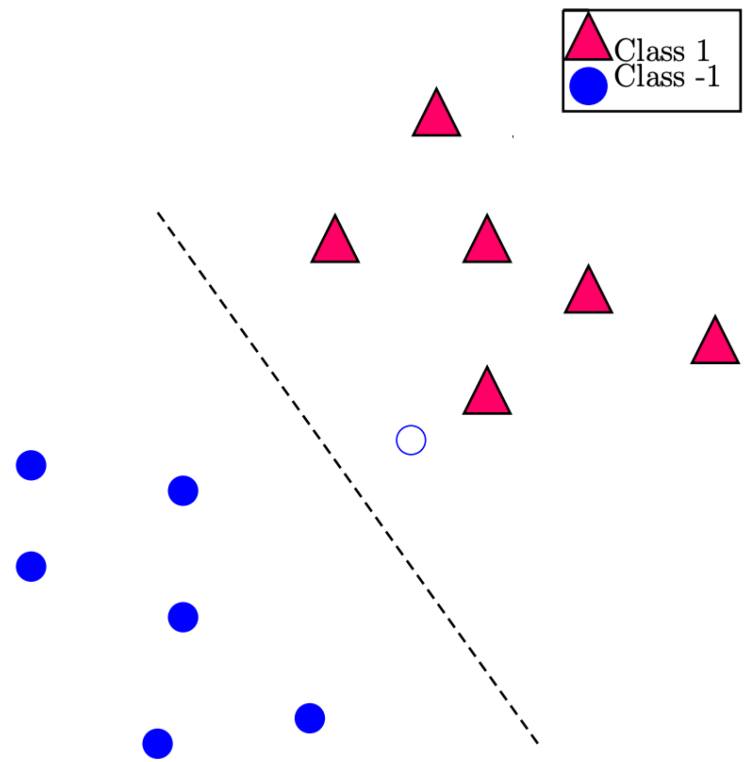


Outline

- Margin and Support Vector
- Dual Problem
- Soft Margin and Regularization
- Kernel Function
- Support Vector Regression
- Kernel Methods

What is the optimal separating line?





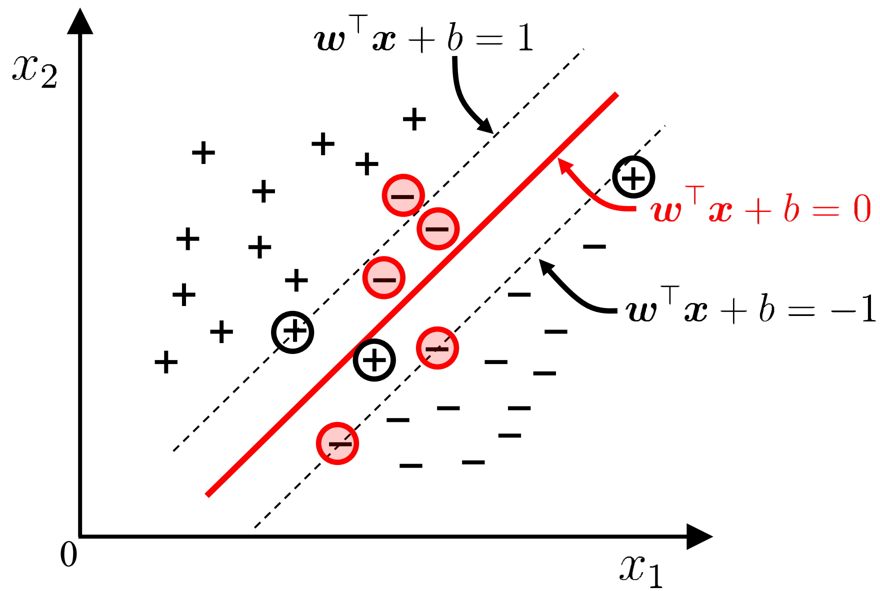
(Both data sets are much better linearly separated if several points are ignored).

Key idea #2: the slack variables

-Q: It is often difficult to find an appropriate kernel function to make the training samples linearly separable in the feature space. Even if we do find such a kernel function, it is hard to tell if it is a result of overfitting.

-A: Allow a support vector machine to make mistakes on a few samples: *soft margin.*

Instances violating the constraint

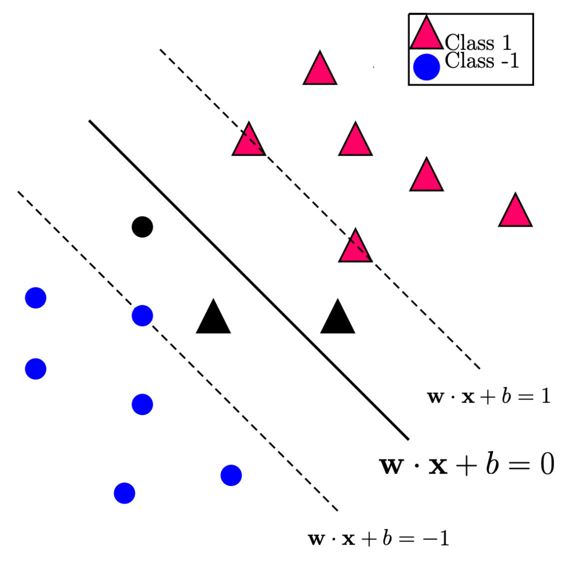


Key idea #2: the slack variables

To find a linear boundary with a large margin, we must allow violations of the constraint $y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1$.

That is, we allow a few points to fall within the margin. They will satisfy

$$y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) < 1$$



There are two cases:

- $y_i = +1$: $\boldsymbol{w} \cdot \boldsymbol{x}_i + b < 1$;
- $y_i = -1$: $\boldsymbol{w} \cdot \boldsymbol{x}_i + b > -1$.

Key idea #2: the slack variables

Formally, we introduce *slack variables* $\xi_1, \ldots, \xi_n \geq 0$ (one for each sample) to allow for exceptions:

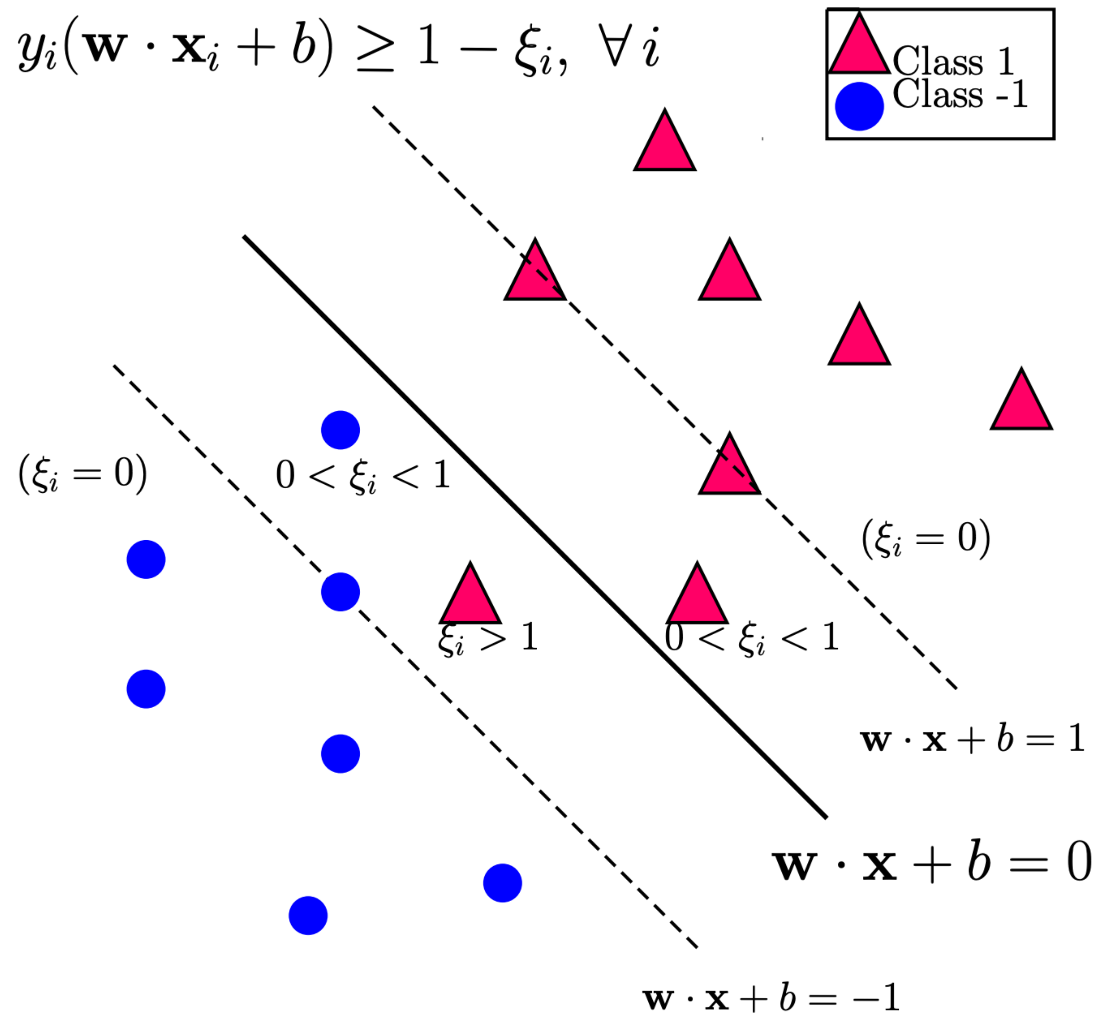
$$y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1 - \xi_i, \qquad \forall i$$

where $\xi_i = 0$ for the points in ideal locations, and $\xi_i > 0$ for the violations (chosen precisely so that the equality will hold true):

- $0 < \xi_i < 1$: Still on correct side of hyperplane but within the margin
- $\xi_i > 1$: Already on wrong side of hyperplane

We say that such an SVM has a ***soft margin*** to distinguish from the previous hard margin.

Key idea #2: the slack variables



Introducing Slack Variables

Because we want most of the points to be in ideal locations, we incorporate the slack variables into the objective function as follows

$$\min_{\mathbf{w},b,\vec{\xi}} \frac{1}{2}\|\mathbf{w}\|_2^2 + C \cdot \underbrace{\sum_i 1_{\xi_i>0}}_{\#\ \text{exceptions}}$$

where $C > 0$ is a regularization constant:

- Larger C leads to fewer exceptions (smaller margin, possible overfitting).
- Smaller C tolerates more exceptions (larger margin, possible underfitting).

Clearly, there must be a tradeoff between margin and #exceptions when selecting the optimal C (often based on cross validation).

ℓ_1 relaxation of the penalty term

The discrete nature of the penalty term on previous slide, $\sum_i 1_{\xi_i > 0} = ||\vec{\xi}||_0$, makes the problem intractable.

A common strategy is to replace the ℓ_0 penalty with a ℓ_1 penalty: $\sum_i \xi_i = ||\vec{\xi}||_1$, resulting in the following full problem

$$\min_{\mathbf{w},b,\vec{\xi}} \frac{1}{2}\|\mathbf{w}\|_2^2 + C \cdot \sum_i \xi_i$$

$$\text{subject to} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i \text{ and } \xi_i \geq 0 \text{ for all } i.$$

Remarks:
(1) Also a quadratic program with linear ineq. constraints (just more variables): $y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) + \xi_i \geq 1$.

The Lagrange dual problem

The associated Lagrange function is

$$L(\mathbf{w}, b, \vec{\xi}, \vec{\lambda}, \vec{\mu}) = \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\lambda_i(y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1 + \xi_i) - \sum_{i=1}^{n}\mu_i\xi_i$$

To find the dual problem we need to fix $\vec{\lambda}$, $\vec{\mu}$ and maximize over $\boldsymbol{w}, b, \vec{\xi}$:

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum \lambda_i y_i \mathbf{x}_i = 0$$

$$\frac{\partial L}{\partial b} = \sum \lambda_i y_i = 0$$

$$\frac{\partial L}{\partial \xi_i} = C - \lambda_i - \mu_i = 0, \quad \forall\, i$$

The Lagrange dual problem

This yields the Lagrange dual function

$$L^*(\vec{\lambda}, \vec{\mu}) = \sum \lambda_i - \frac{1}{2} \sum \lambda_i \lambda_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j, \qquad \text{where}$$

$$\lambda_i \geq 0,\ \ \mu_i \geq 0,\ \ \lambda_i + \mu_i = C,\ \ \text{and} \ \sum \lambda_i y_i = 0.$$

The dual problem would be to maximize L^* over $\vec{\lambda}, \vec{\mu}$ subject to the constraints.

Since L^* is constant with respect to the μ_i, we can eliminate them to obtain a reduced dual problem:

$$\max_{\lambda_1, \ldots, \lambda_n} \sum \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\text{subject to} \quad \underbrace{0 \leq \lambda_i \leq C}_{\text{box constraints}} \ \ \text{and} \ \sum \lambda_i y_i = 0.$$

What changed?

What about the KKT conditions?

The KKT conditions are the following

$$\mathbf{w} = \sum \lambda_i y_i \mathbf{x}_i, \quad \sum \lambda_i y_i = 0, \quad \lambda_i + \mu_i = C$$

$$\lambda_i(y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 + \xi_i) = 0, \quad \mu_i \xi_i = 0$$

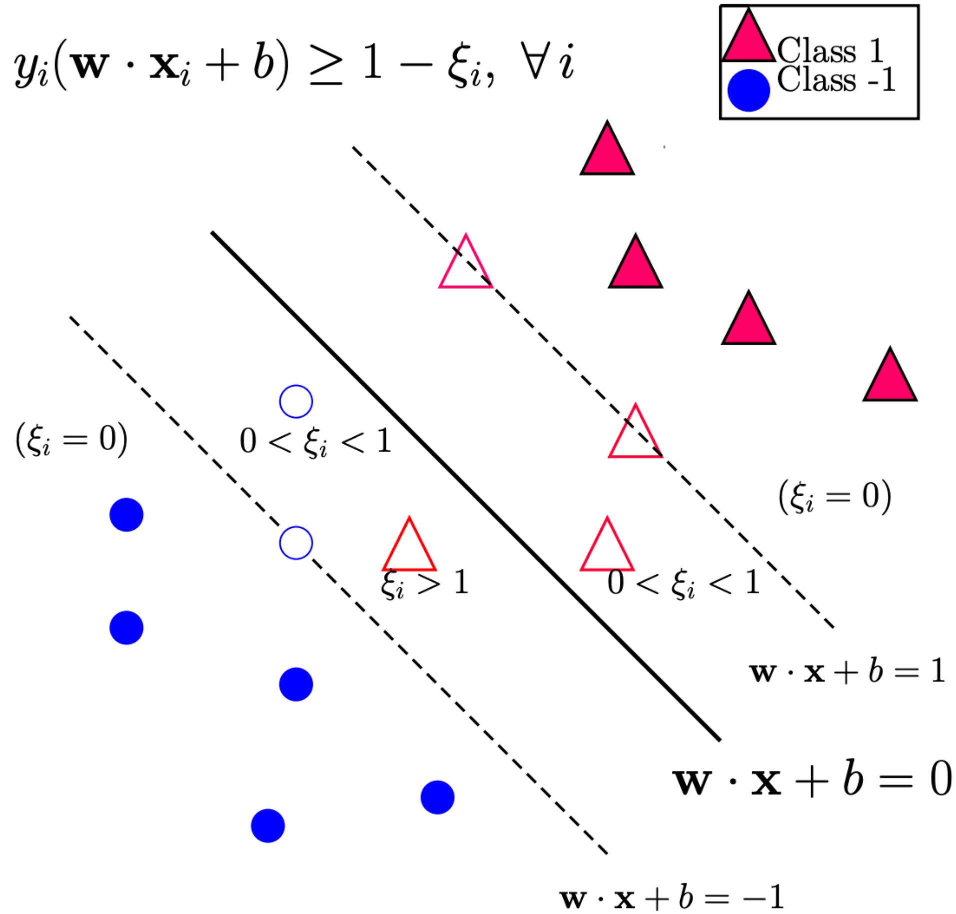
$$\lambda_i \geq 0, \quad \mu_i \geq 0$$

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

We see that

- The optimal **w** has the same formula: $\boldsymbol{w} = \sum \lambda_i y_i \boldsymbol{x}_i$.
- Any point with $\lambda_i > 0$ and correspondingly $y_i(\boldsymbol{w} \cdot \boldsymbol{x} + b) = 1 - \xi_i$ is a support vector (not just those on the margin boundary $\boldsymbol{w} \cdot \boldsymbol{x} + b = \pm 1$).

To find b

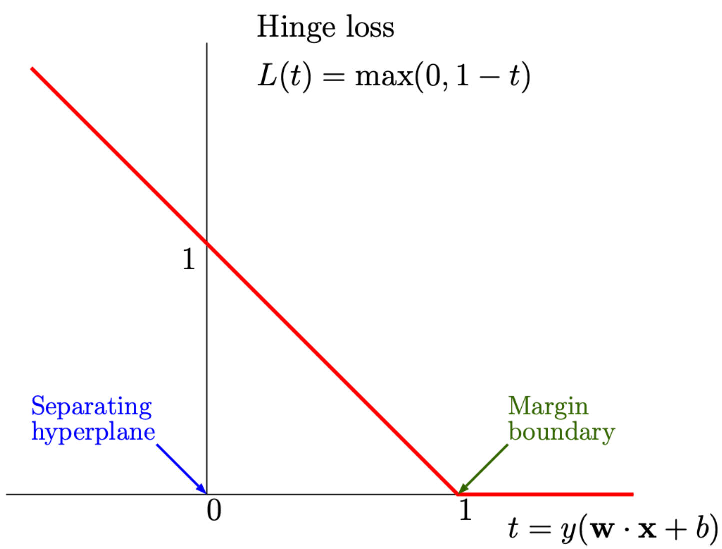


To find b, choose any support vector $\boldsymbol{x}_i$ with $0 < \lambda_i < C$ (which implies that $\mu_i > 0$ and $\xi_i = 0$), and use the formula $b = \frac{1}{y_i} - \boldsymbol{w} \cdot \boldsymbol{x}_i$.

ℓ_1 relaxation of the penalty term

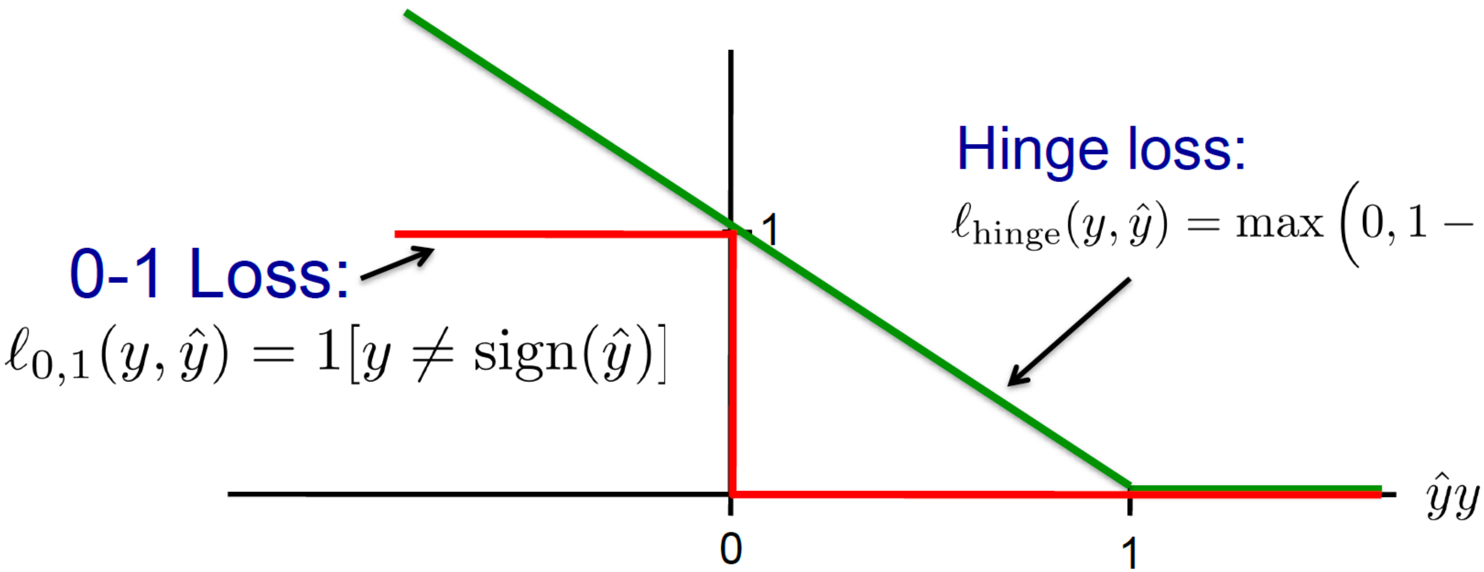
The primal problem may be rewritten as an unconstrained optimization problem

$$\min_{\mathbf{w},b} \quad \underbrace{\frac{1}{2}\|\mathbf{w}\|_2^2}_{\text{regularization}} \quad + C \cdot \underbrace{\sum_{i=1}^{n} \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i + b))}_{\text{hinge loss}}$$



Sub-gradient descent
Projected Gradient Descent

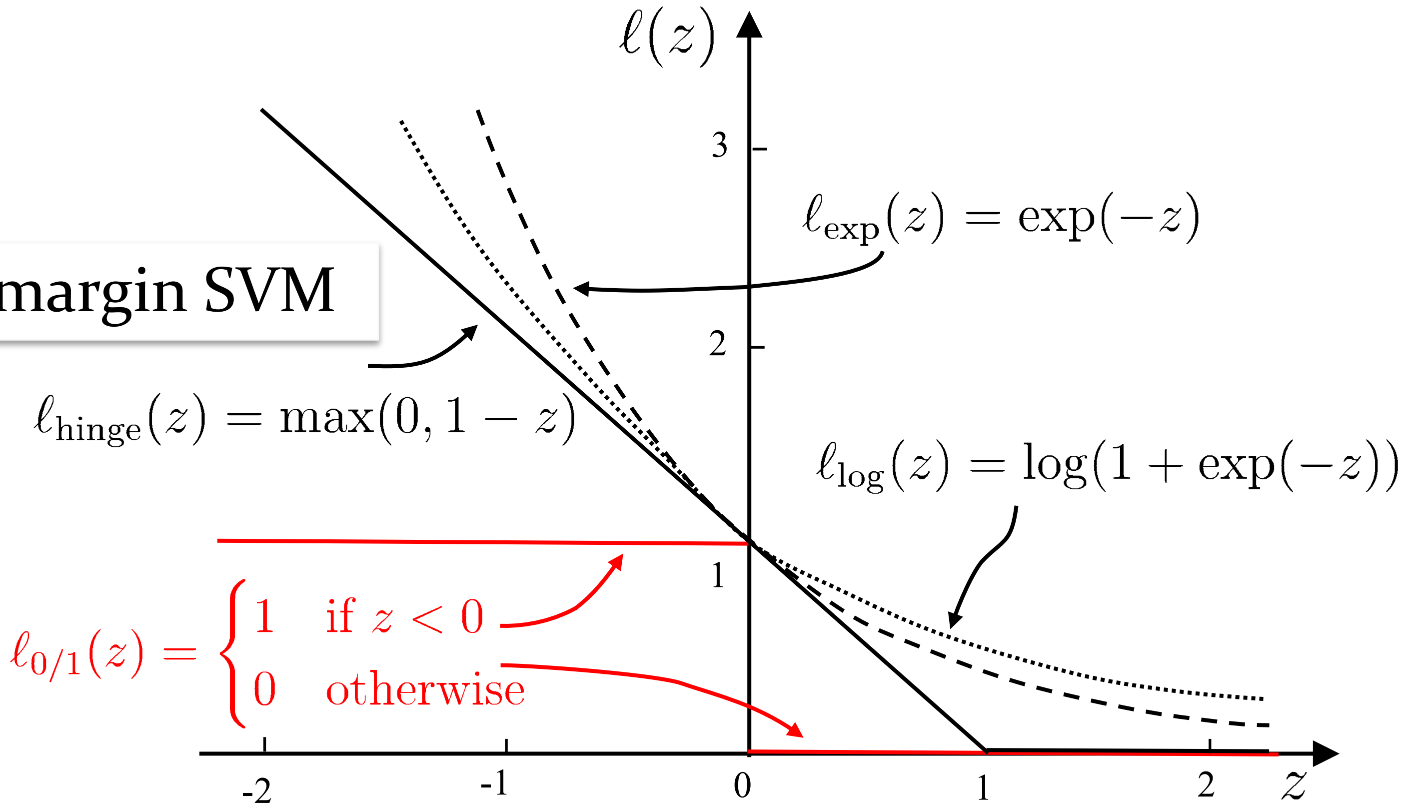
Hinge loss vs. 0/1 loss



Hinge loss upper bounds 0/1 loss!

It is the tightest convex upper bound on the 0/1 loss

Surrogate loss functions



Surrogate loss functions have nice mathematical properties, e.g., convex, continuous, and are upper bound of 0/1 loss function

Regularization

- General form of SVM models:

$$\min_f \quad \Omega(f) + C\sum_{i=1}^{m} l(f(\boldsymbol{x}_i), y_i)$$

Structural risk, representing some properties of the model

Empirical risk, describing how well the model matches the training data

- Other learning models can be derived by substituting the above components
  - Logistic Regression
  - LASSO
  - ……

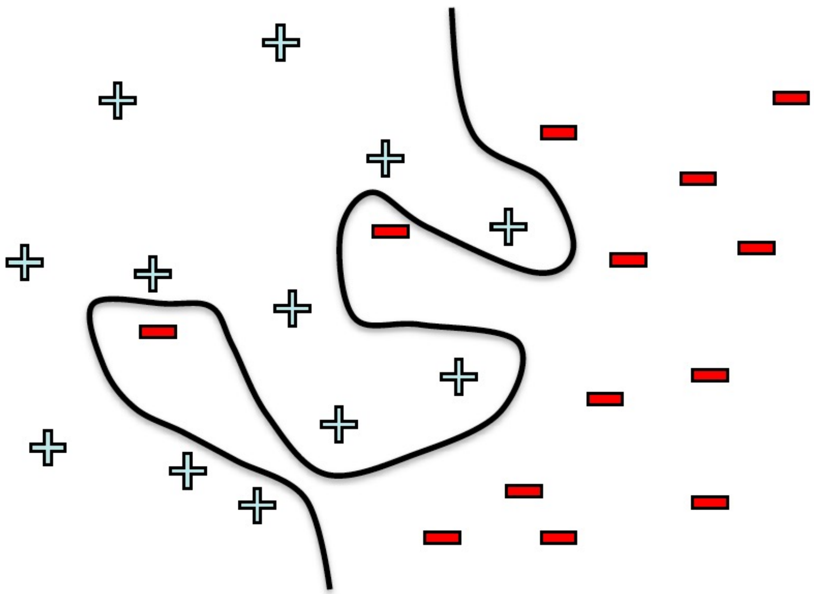
Primal and Dual Problem

维度	原问题	对偶问题
变量数量	$d+1$（特征维度）	n（样本数）
适用数据规模	$n \gg d$（大规模样本低维数据）	$d \gg n$（小样本高维数据）
非线性处理	需显式特征工程（不直接支持核技巧）	天然支持核技巧
内存消耗	$O(d)$	$O(n^2)$（核矩阵存储）
求解效率	低维高效，高维复杂	小样本高效，大样本受限
典型算法	梯度下降、Pegasos	SMO、内点法、QP求解器

Outline

- Margin and Support Vector
- Dual Problem
- Soft Margin and Regularization
- Kernel Function
- Support Vector Regression
- Kernel Methods

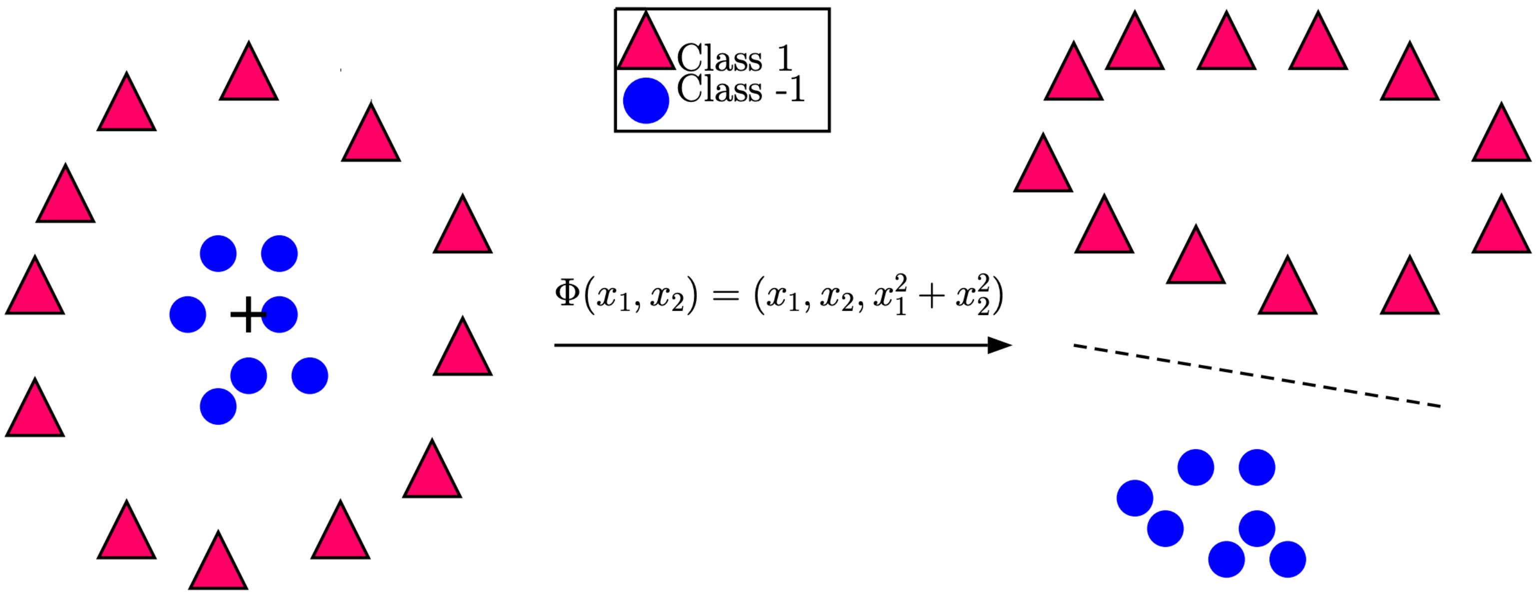
What if the data is not linearly separable?



Use features of features of features of features....

$$\phi(x) = \begin{pmatrix} x^{(1)} \\ \ldots \\ x^{(n)} \\ x^{(1)}x^{(2)} \\ x^{(1)}x^{(3)} \\ \ldots \\ e^{x^{(1)}} \\ \ldots \end{pmatrix}$$

Use Feature Map

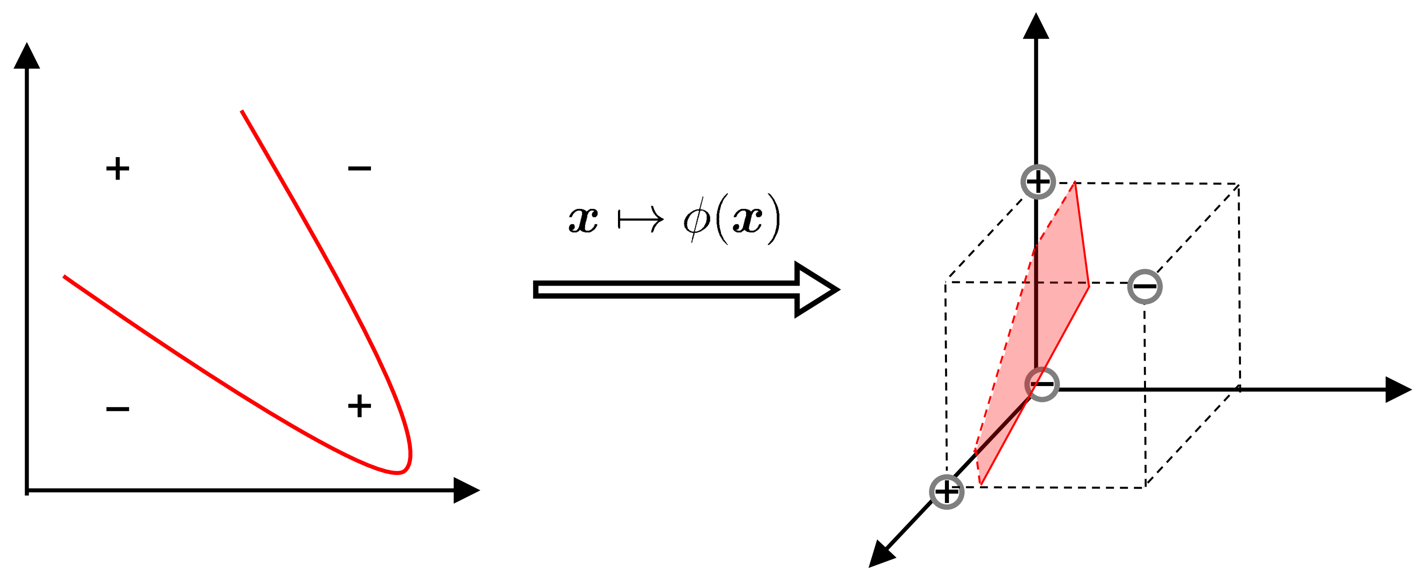


Feature space can get really large really quickly!

Key idea #3: the kernel trick

- High dimensional feature spaces at no extra cost!
- Map the samples from the original feature space to a

higher dimensional feature space. That way the samples become linearly separable.



Kernel SVM

Let $\phi(\boldsymbol{x})$ denote the mapped feature vector of $\boldsymbol{x}$, the separating hyperplane $f(\boldsymbol{x}) = \boldsymbol{w}^\top \phi(\boldsymbol{x}) + b$ can be expressed as

Primal Problem

$$\min_{\boldsymbol{w},b} \quad \frac{1}{2}\|\boldsymbol{w}\|^2$$
$$\text{s.t.} \quad y_i(\boldsymbol{w}^\top \phi(\boldsymbol{x}_i) + b) \geq 1, \ \ i = 1, 2, \ldots, m.$$

Dual Problem

$$\min_{\boldsymbol{\alpha}} \quad \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m} \alpha_i\alpha_j y_i y_j \boxed{\phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}_j)} - \sum_{i=1}^{m} \alpha_i$$
$$\text{s.t.} \quad \sum_{i=1}^{m} \alpha_i y_i = 0, \ \ \alpha_i \geq 0, \ \ i = 1, 2, \ldots, m.$$

Prediction

$$f(\boldsymbol{x}) = \boldsymbol{w}^\top \phi(\boldsymbol{x}) + b = \sum_{i=1}^{m} \alpha_i y_i \boxed{\phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x})} + b$$

Kernel function

- Basic idea： design kernel function instead of kernel mapping explicitly

$$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}_j)$$

- Mercer's theorem (sufficient, nonessential)： if only the corresponding kernel matrix of a symmetric function is positive-definite, it can act as a kernel function. *(analogous to the definition of a positive-semidefinite matrix)*
- Common kernel functions：

Name	Expression	Parameters
Linear kernel	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \boldsymbol{x}_i^{\mathrm{T}} \boldsymbol{x}_j$	
Polynomial kernel	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = (\boldsymbol{x}_i^{\mathrm{T}} \boldsymbol{x}_j)^d$	$d \geqslant 1$ is the degree of the polynomial.
Gaussian kernel	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp\left(-\frac{\lVert\boldsymbol{x}_i - \boldsymbol{x}_j\rVert^2}{2\sigma^2}\right)$	$\sigma > 0$ is the width of the Gaussian kernel.
Laplacian kernel	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp\left(-\frac{\lVert\boldsymbol{x}_i - \boldsymbol{x}_j\rVert}{\sigma}\right)$	$\sigma > 0$.
Sigmoid kernel	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \tanh(\beta \boldsymbol{x}_i^{\mathrm{T}} \boldsymbol{x}_j + \theta)$	tanh is the hyperbolic tangent function, $\beta > 0$, $\theta < 0$.

What are good kernel functions?

- Linear kernel
  - $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)\phi(\boldsymbol{x}_j) = \boldsymbol{x}_i \cdot \boldsymbol{x}_j$
- Polynomial
  - $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = (\boldsymbol{x}_i \cdot \boldsymbol{x}_j + 1)^{\mathrm{n}}$
- Gaussian (also called Radial Basis Function, or RBF)
  - $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = e^{\frac{\left\|x_i - x_j\right\|^2}{2\sigma^2}}$
- ...

Kernel algebra

kernel composition	**feature composition**
a) $k(\mathbf{x},\mathbf{v}) = k_a(\mathbf{x},\mathbf{v}) + k_b(\mathbf{x},\mathbf{v})$	$\boldsymbol{\phi}(\mathbf{x}) = (\boldsymbol{\phi}_a(\mathbf{x}), \boldsymbol{\phi}_b(\mathbf{x})),$
b) $k(\mathbf{x},\mathbf{v}) = f k_a(\mathbf{x},\mathbf{v}),\ f > 0$	$\boldsymbol{\phi}(\mathbf{x}) = \sqrt{f}\boldsymbol{\phi}_a(\mathbf{x})$
c) $k(\mathbf{x},\mathbf{v}) = k_a(\mathbf{x},\mathbf{v}) k_b(\mathbf{x},\mathbf{v})$	$\phi_m(\mathbf{x}) = \phi_{ai}(\mathbf{x})\phi_{bj}(\mathbf{x})$
d) $k(\mathbf{x},\mathbf{v}) = \mathbf{x}^T A \mathbf{v}$, A positive semi-definite	$\boldsymbol{\phi}(\mathbf{x}) = L^T\mathbf{x}$, where $A = LL^T$.
e) $k(\mathbf{x},\mathbf{v}) = f(\mathbf{x})f(\mathbf{v})k_a(\mathbf{x},\mathbf{v})$	$\phi(\mathbf{x}) = f(\mathbf{x})\phi_a(\mathbf{x})$

Quadratic kernel

$$
\begin{aligned}
k(\mathbf{x},\mathbf{z}) &= (\mathbf{x}^T\mathbf{z}+c)^2 = \left(\sum_{j=1}^{n} x^{(j)}z^{(j)} + c\right)\left(\sum_{\ell=1}^{n} x^{(\ell)}z^{(\ell)} + c\right) \\
&= \sum_{j=1}^{n}\sum_{\ell=1}^{n} x^{(j)}x^{(\ell)}z^{(j)}z^{(\ell)} + 2c\sum_{j=1}^{n} x^{(j)}z^{(j)} + c^2 \\
&= \sum_{j,\ell=1}^{n} (x^{(j)}x^{(\ell)})(z^{(j)}z^{(\ell)}) + \sum_{j=1}^{n}(\sqrt{2c}x^{(j)})(\sqrt{2c}z^{(j)}) + c^2,
\end{aligned}
$$

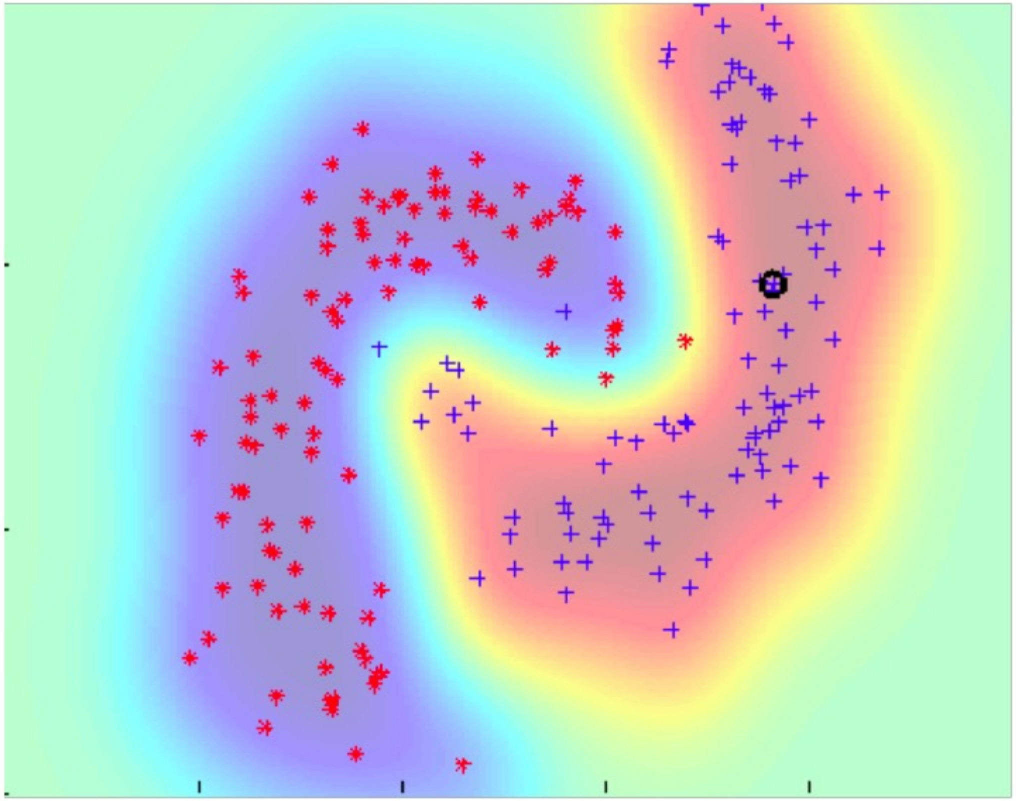
Feature mapping given by:

$$
\mathbf{\Phi}(\mathbf{x}) = [x^{(1)2}, x^{(1)}x^{(2)}, \ldots, x^{(3)2}, \sqrt{2c}x^{(1)}, \sqrt{2c}x^{(2)}, \sqrt{2c}x^{(3)}, c]
$$

Gaussian kernel (RBF)

$$K(\vec{u}, \vec{v}) = \exp\left(-\frac{||\vec{u} - \vec{v}||_2^2}{2\sigma^2}\right)$$

$$y \leftarrow \text{sign}\left[\sum_i \alpha_i y_i \exp\left(-\frac{\|\vec{x} - \vec{x}_i\|_2^2}{2\sigma^2}\right) + b\right]$$



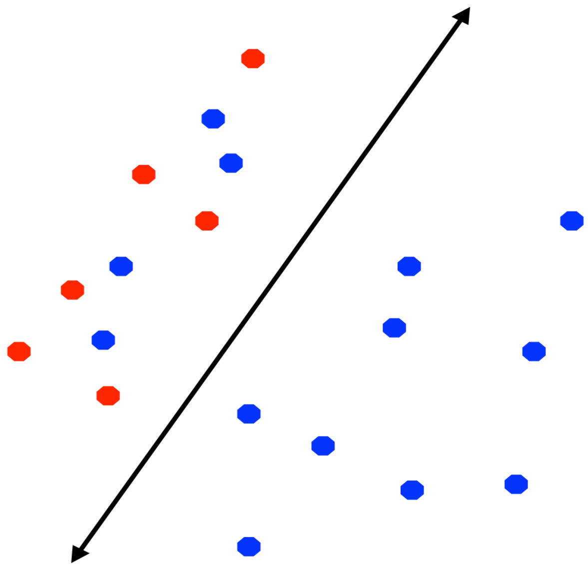
$$\psi_{\mathrm{RBF}} : \mathbb{R}^n \to \mathbb{R}^\infty$$

Proof?

Hint:
Taylor expansion of exponential function

The feature mapping is infinite dimensional!

How to deal with imbalanced data?



- In many practical applications we may have **imbalanced** data sets
- We may want errors to be equally distributed between the positive and negative classes
- A slight modification to the SVM objective does the trick!

$$N = N_+ + N_-$$

$$\min_{w,b} \quad ||w||_2^2 + \frac{CN}{2N_+} \sum_{j:y_j=+1} \xi_j + \frac{CN}{2N_-} \sum_{j:y_j=-1} \xi_j$$

Class-specific weighting of the slack variables

Overfitting?

- Huge feature space with kernels: should we worry about overfitting?
  - SVM objective seeks a solution with large margin
    - Theory says that large margin leads to good generalization
  - But everything overfits sometimes!!!
  - Can control by:
    - Setting C
    - Choosing a better Kernel
    - Varying parameters of the Kernel (width of Gaussian, etc.)

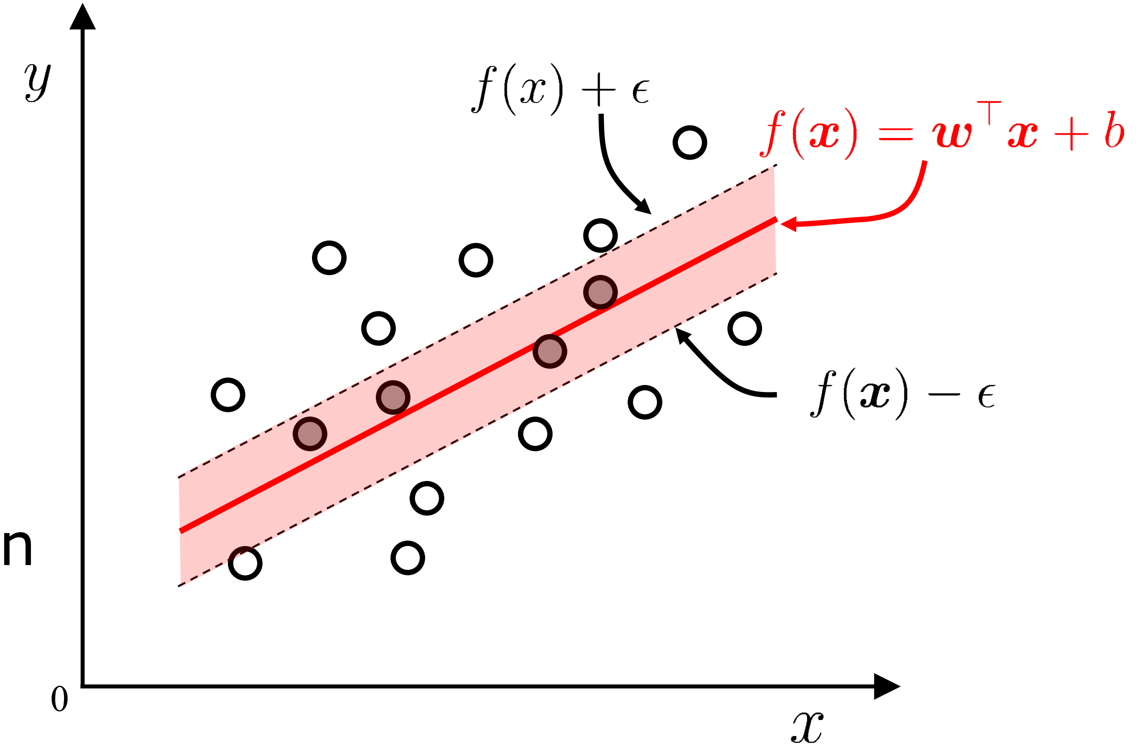
Outline

- Margin and Support Vector
- Dual Problem
- Soft Margin and Regularization
- Kernel Function
- Support Vector Regression
- Kernel Methods

Support vector regression

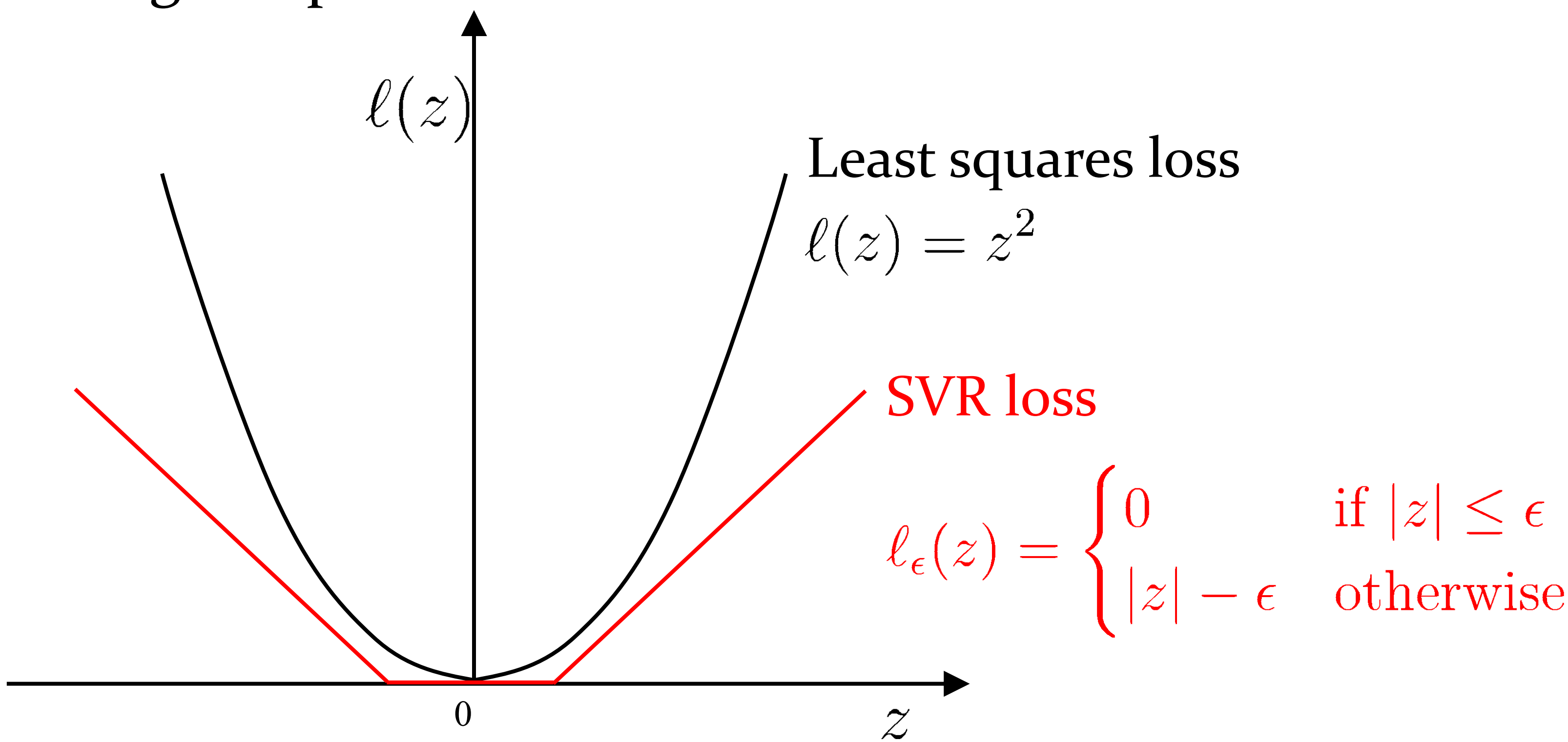
Allows an error 2ϵ between model output and ground truth

2ϵ region



Loss function

Training samples falling within 2ϵ region are considered as correctly predicted, that is, no loss. The solution of SVR is sparse since the support vectors are only a subset of the training samples.



Formulation

Primal Problem

$$\begin{aligned} \min_{\boldsymbol{w}, b, \xi_i, \hat{\xi}_i} \quad & \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{m}(\xi_i + \hat{\xi}_i) \\ \text{s.t.} \quad & y_i - \boldsymbol{w}^\top \phi(\boldsymbol{x}_i) - b \leq \epsilon + \xi_i, \\ & y_i - \boldsymbol{w}^\top \phi(\boldsymbol{x}_i) - b \geq -\epsilon - \hat{\xi}_i, \\ & \xi_i \geq 0,\ \hat{\xi}_i \geq 0,\ i = 1, 2, \ldots, m. \end{aligned}$$

Dual Problem

$$\begin{aligned} \min_{\boldsymbol{\alpha}, \hat{\boldsymbol{\alpha}}} \quad & \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}(\alpha_i - \hat{\alpha}_i)(\alpha_j - \hat{\alpha}_j)\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) + \sum_{i=1}^{m}(\alpha_i(\epsilon - y_i) + \hat{\alpha}_i(\epsilon + y_i)) \\ \text{s.t.} \quad & \sum_{i=1}^{m}(\alpha_i - \hat{\alpha}_i) = 0, \\ & 0 \leq \alpha_i \leq C,\ 0 \leq \hat{\alpha}_i \leq C. \end{aligned}$$

Prediction $f(\boldsymbol{x}) = \boldsymbol{w}^\top \phi(\boldsymbol{x}) + b = \sum_{i=1}^{m}(\hat{\alpha}_i - \alpha_i)y_i\kappa(\boldsymbol{x}_i, \boldsymbol{x}) + b$

Outline

- Margin and Support Vector
- Dual Problem
- Soft Margin and Regularization
- Kernel Function
- Support Vector Regression
- Kernel Methods

Representer theorem

SVM $$f(\boldsymbol{x}) = \boldsymbol{w}^\top \phi(\boldsymbol{x}) + b = \sum_{i=1}^{m} \alpha_i y_i \kappa(\boldsymbol{x}_i, \boldsymbol{x}) + b$$

SVR $$f(\boldsymbol{x}) = \boldsymbol{w}^\top \phi(\boldsymbol{x}) + b = \sum_{i=1}^{m} (\hat{\alpha}_i - \alpha_i) y_i \kappa(\boldsymbol{x}_i, \boldsymbol{x}) + b$$

Conclusion: The learned models of SVM and SVR can be expressed as a linear combination of the kernel functions.

A more generalized conclusion(representer theorem): for any increasing function Ω and any non-negative loss function l, the optimization problem

$$\min_{h \in \mathbb{H}} \quad F(h) = \Omega(\|h\|_{\mathbb{H}}) + l(h(\boldsymbol{x}_1), \ldots, h(\boldsymbol{x}_m))$$

Solution can be written in the form of $h^* = \sum_{i=1}^{m} \alpha_i \kappa(\cdot, \boldsymbol{x}_i)$

Summary

- Unusual choice of separation strategy:
  - Maximize “street” between groups
- Attack maximization problem:
  - Lagrange multipliers + hairy mathematics
- New problem is a quadratic minimization
  - Susceptible to fancy numerical methods
- Result depends on dot products only
  - Enables use of kernel methods

Credits

- The flow of this SVM lecture goes to
  - Patrick Winston, Professor of Artificial Intelligence
  - Director of MIT Artificial Intelligence Lab (1992-1997)
  - Taught 6.034: Artificial Intelligence

https://ocw.mit.edu/courses/6-034-artificial-intelligence-fall-2010/

1943-2019

Take Home Message

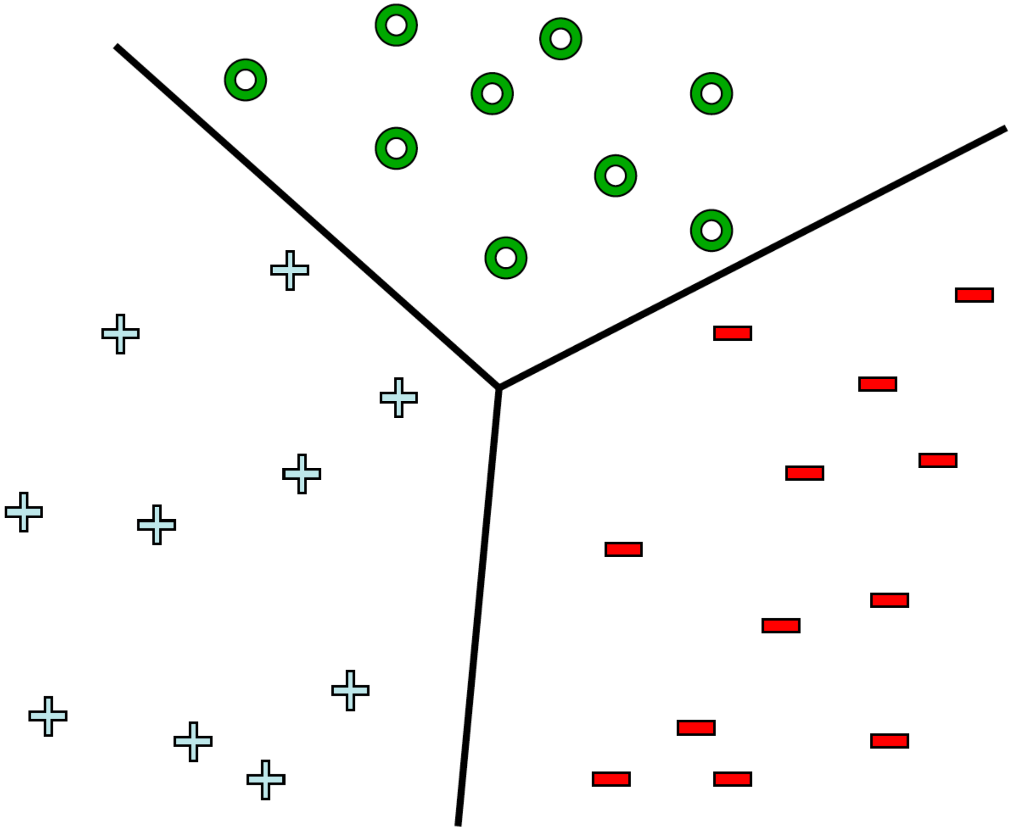
- The “large margin” idea of SVM
- Dual problem and the sparsity of the solution
- Solving linear inseparable problems by projecting to high-dimensional space
- Solving linear inseparable problems in the feature space by introducing “soft margin”
- Utilizing the idea of support vectors into regression tasks and get SVR
- Extending kernel methods to other learning models

Mature SVM packages

- LIBSVM
  http://www.csie.ntu.edu.tw/~cjlin/libsvm/
- LIBLINEAR
  http://www.csie.ntu.edu.tw/~cjlin/liblinear/
- SVM^{light}、SVM^{perf}、SVM^{struct}
  http://svmlight.joachims.org/svm_struct.html
- Scikit-learn
  http://scikit-learn.org/stable/modules/svm.html

Multi-class Classification

How do we do multi-class classification?



Multiclass Classification

- Multiclass Classification learning methods
  - Some binary classification methods can be directly extended to accommodate multiclass cases
  - Apply some strategies to solve multiclass classification problems with any existing binary classification methods （more general）
    - Decompose the problem and then train a binary classifier for each divided binary classification problem
    - Ensemble the outputs collected from all binary classifiers into the final multiclass predictions
- Dividing strategies
  - One vs. One (OvO)
  - One vs. Rest (OvR)
  - Many vs. Many (MvM)

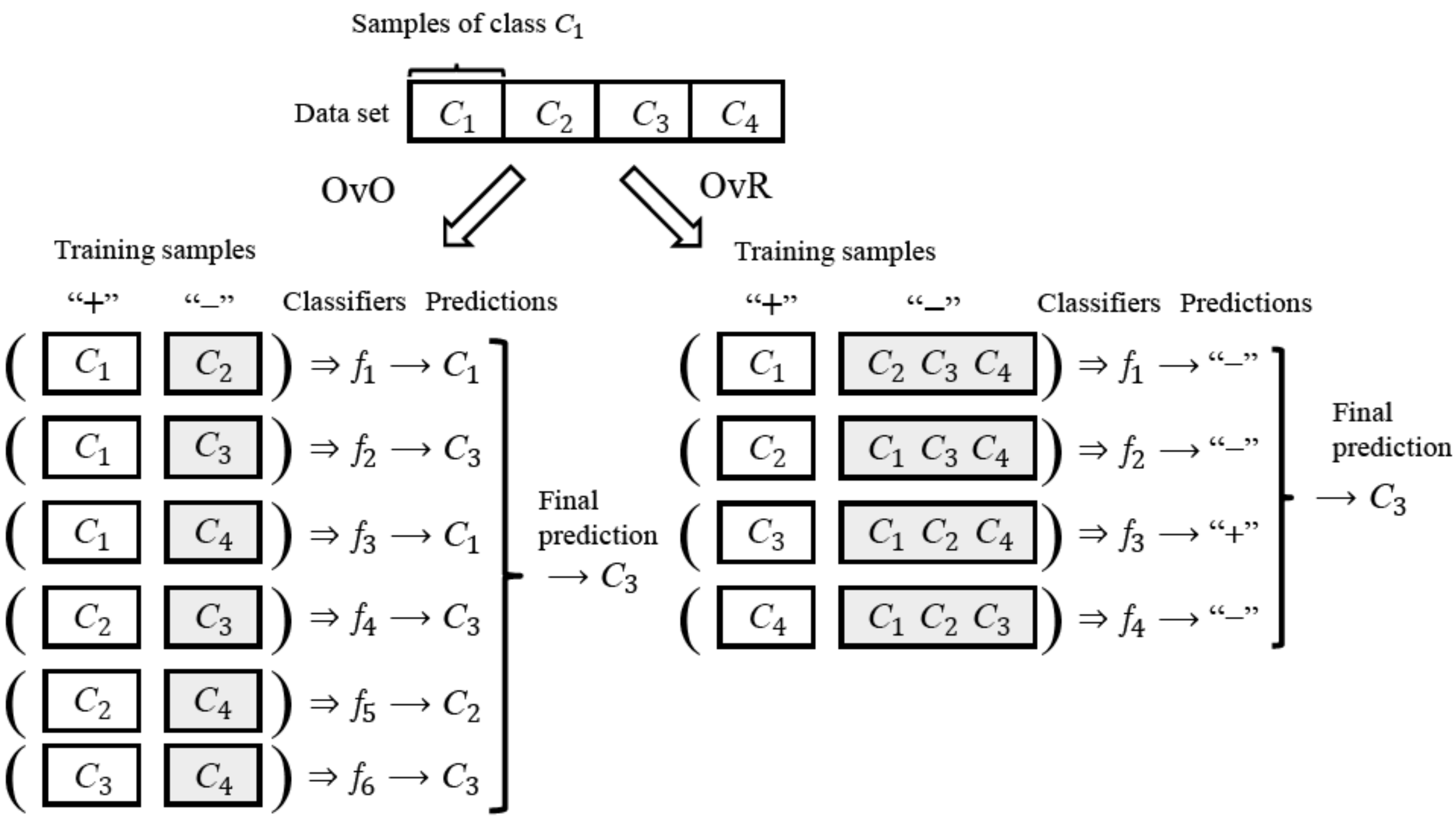
Multiclass Classification - OvO

- In the decomposing phase
  - puts the N classes into pairs
    - $N(N-1)/2$ binary classification tasks
  - trains a classifier for each task
    - $N(N-1)/2$ classifiers
- In the testing phase
  - a new sample is classified by all classifiers
    - $N(N-1)/2$ classification outputs
  - the final prediction can be made via voting
    - the predicted class is the one received the most votes

Multiclass Classification - OvR

- In the decomposing phase
  - consider each class as positive in turn, and the rest classes are considered as negative
    - N binary classification tasks
  - trains a classifier for each task
    - N classifiers
- In the testing phase
  - a new sample is classified by all classifiers
    - N classification outputs
  - the prediction confidences are usually assessed
    - the class with the highest confidence is used as the classification result

Multiclass Classification – A comparison between OvO and OvR



Multiclass Classification – A comparison between OvO and OvR

OvO

- Train $N(N-1)/2$ classifiers， the memory and testing time costs are often higher
- Each classifier uses only samples of two classes. Hence, the computational cost of training OvO is lower

OvR

- Train N classifiers， the memory and testing time costs are often lower
- Each classifier uses all training samples. Hence, the computational cost of training OvO is higher

As for the prediction performance, it depends on the specific data distribution, and in most cases, the two methods have similar performance.

Recent Progress

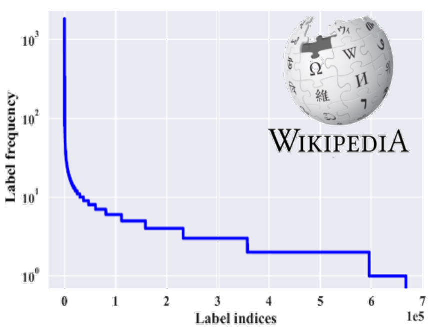
Class Imbalance Problem

- **Deep Learning**

enablers →

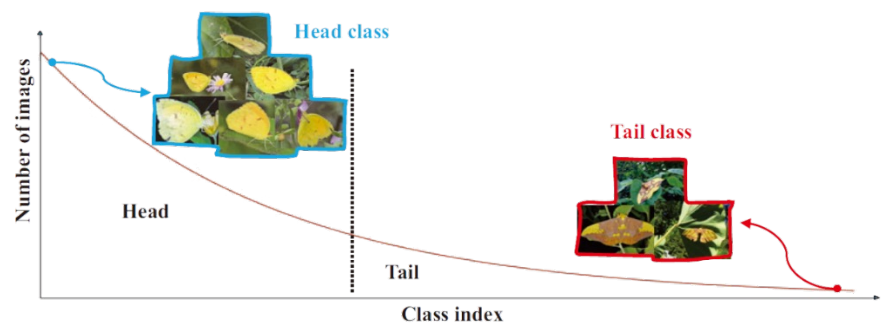
- ➢ Faster computers
- ➢ Algorithmic improvements
- ➢ Access to large amounts of data

- **Real-world class-imbalanced (long-tailed) class distribution**



Zipf's law

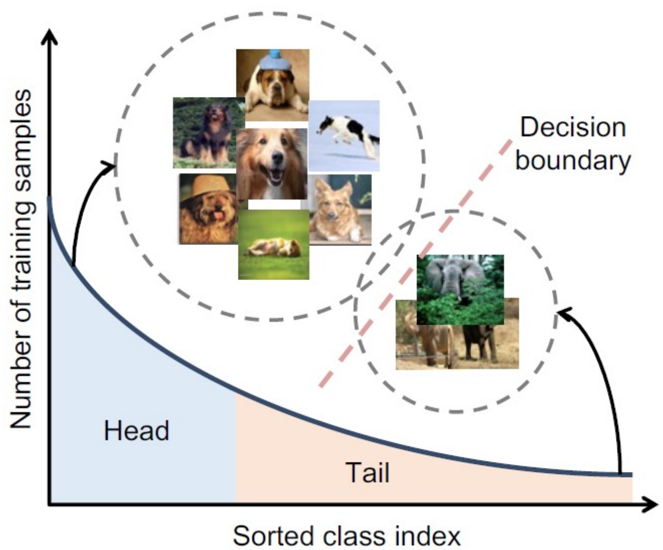
(Power law)



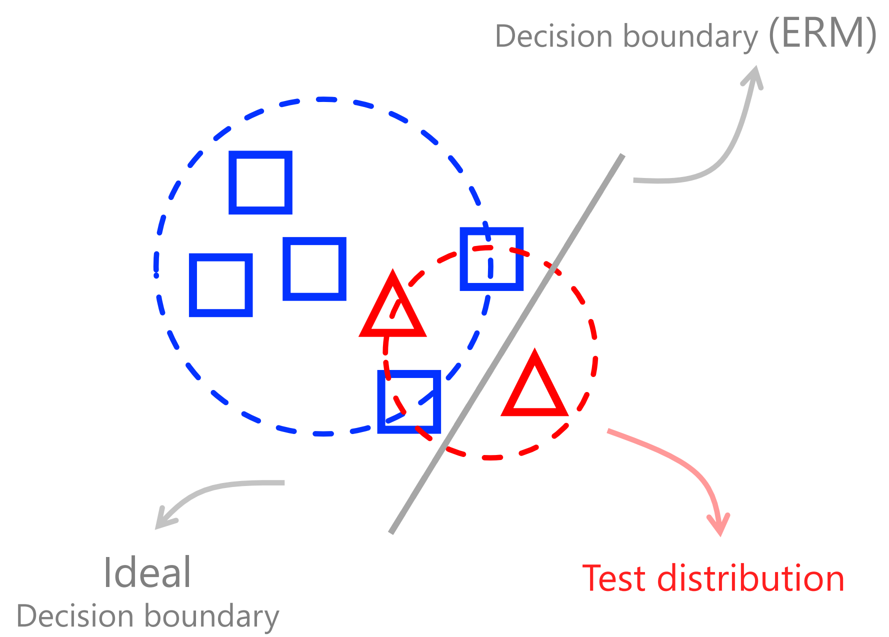
Class Imbalance Problem

Challenge

—— model biased towards head classes



Take an example

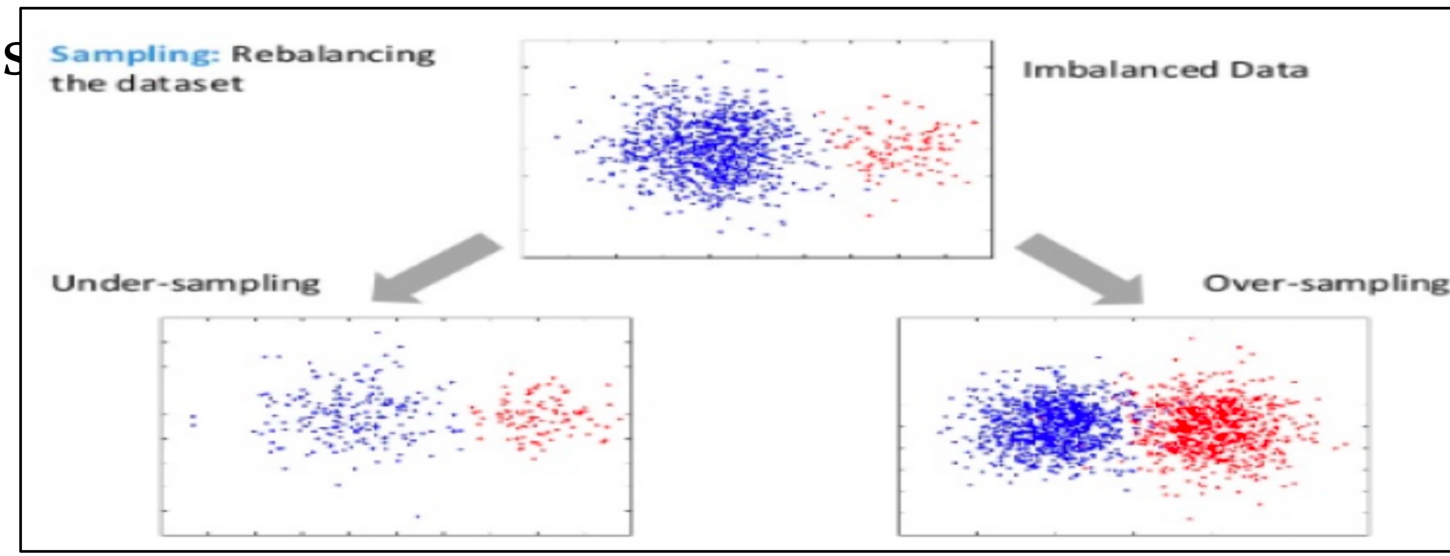


Class Imbalance Problem

- Class imbalance
  - a significantly different number of samples for each class.（the positive class is the minority）

the classes are balanced $\frac{y}{1-y} > 1$ ⇒ $\frac{y}{1-y} > \frac{m^+}{m^-}$ the observed class ratio

- Res

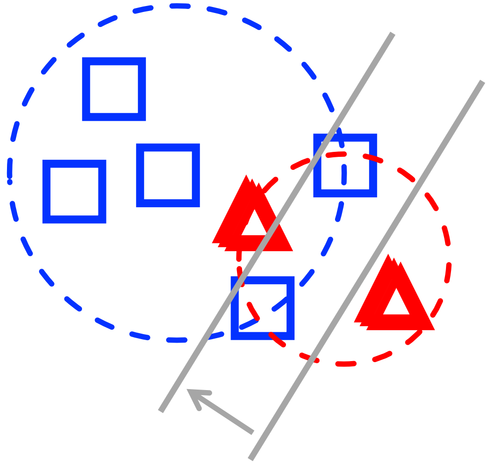


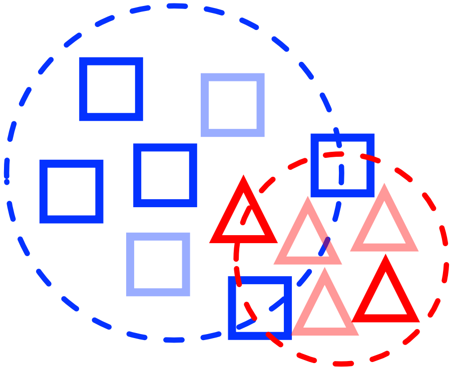
sses

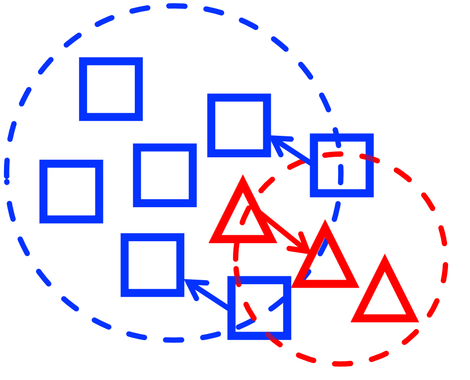
re

Class Imbalance Problem

Class Re-balancing	Data Augmentation	Class-balanced Loss Function

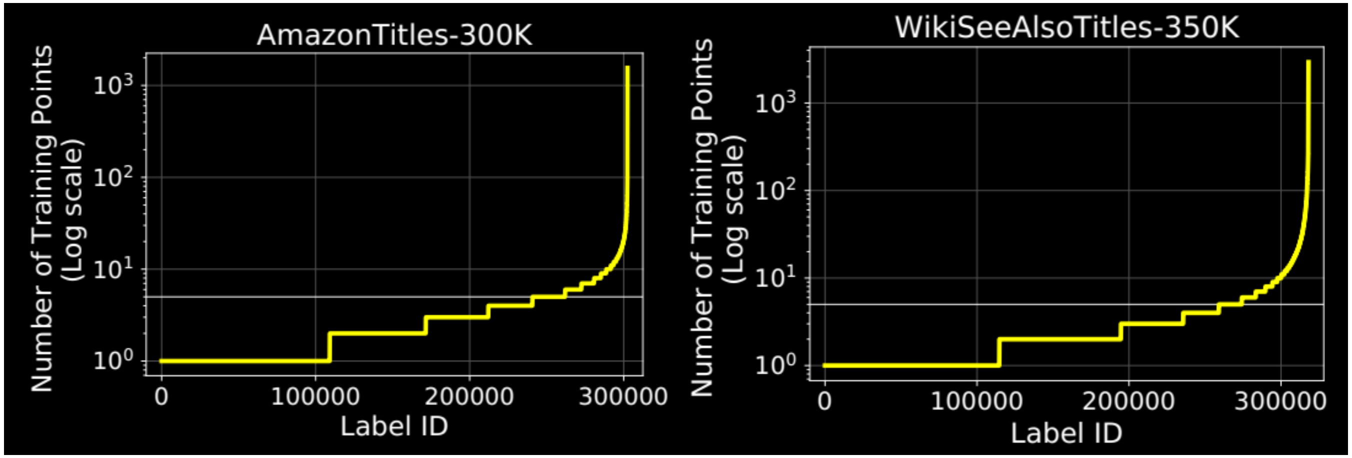




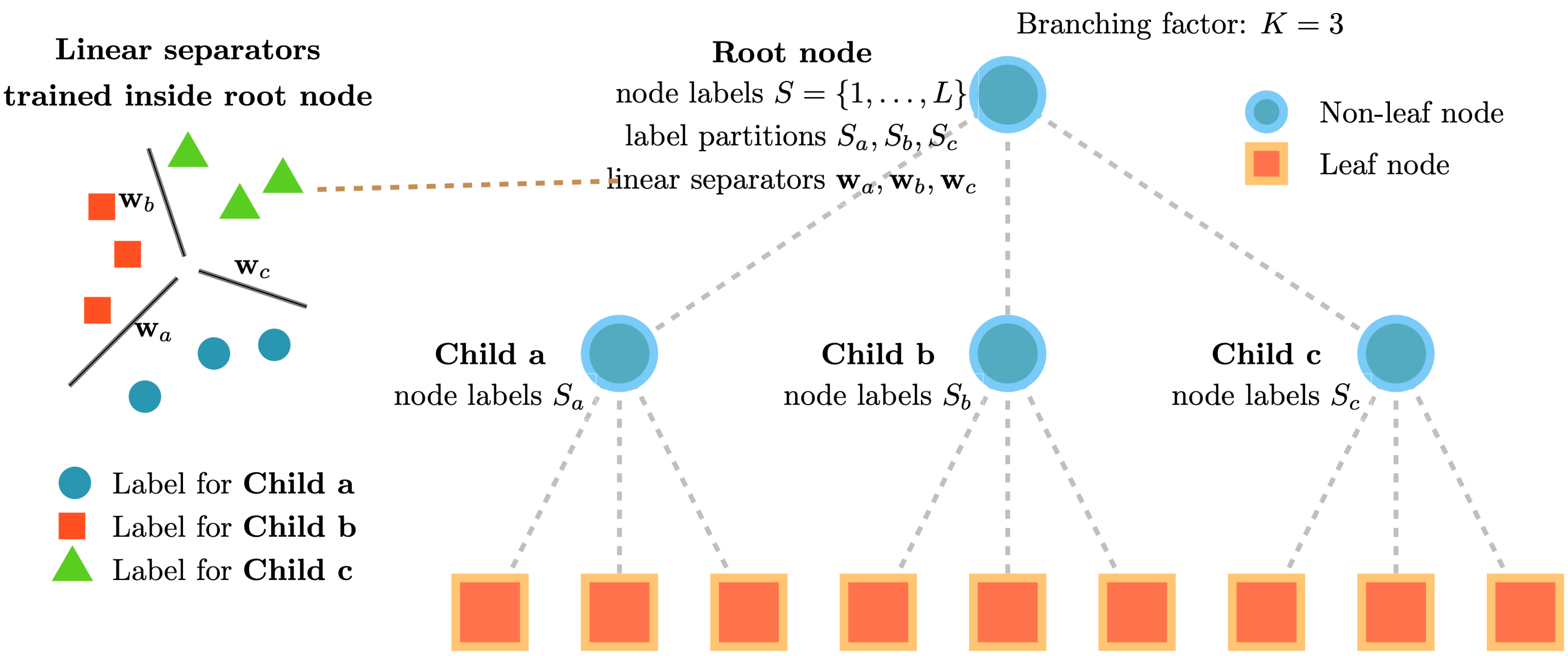


Extreme Classification

	Dataset	# of Train Points	# of Labels	# of Test Points
Benchmark*	LF-AmazonTitles-131K	294,805	131,073	134,835
	LF-WikiSeeAlsoTitles-320K	693,082	312,330	177,515
	LF-AmazonTitles-1.3M	2,248,619	1,305,265	970,237
Bing	LF-P2PTitles-300K	1,366,429	300,000	585,602
	LF-P2PTitles-2M	2,539,009	1,640,898	1,088,146



Label Tree



- recursively partition the set of classes into subsets
- train a multi-class (linear) classifier for inference